ĐẠI HỌC TRÀ VINH

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC: CHUYÊN ĐỀ ASP.NET

HỌC KỲ IV, NĂM HỌC 2025-2026

XÂY DỰNG WEBSITE
QUẢN LÝ KHÁCH SẠN

Giảng viên hướng dẫn:

TS. ĐOÀN PHƯỚC MIỀN

Sinh viên thực hiện:

Họ tên: PHẠM TRUNG THIỆN

MSSV: 170124499

Lớp:DT24TT80161

Vĩnh long, năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Trà Vinh, ngày ..... tháng ..... năm .....

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Trà Vinh, ngày ..... tháng ..... năm .....

Thành viên hội đồng

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trong suốt quá trình học tập, nghiên cứu và thực hiện đồ án môn học với đề tài *"Xây dựng ứng dụng web quản lý và đặt phòng khách sạn HotelManager trên nền tảng ASP.NET Core MVC"*, tôi đã nhận được rất nhiều sự quan tâm, hướng dẫn tận tình và tạo điều kiện thuận lợi từ quý thầy cô cùng bạn bè.

Trước hết, tôi xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến giảng viên hướng dẫn bộ môn — Thầy/TS ĐOÀN PHƯỚC MIỀN. Thầy đã trực tiếp định hướng đề tài, truyền đạt những kiến thức chuyên môn quý báu về công nghệ phát triển ứng dụng web trên nền tảng .NET, đồng thời tận tình góp ý, giải đáp những vướng mắc kỹ thuật để tôi có thể từng bước hoàn thiện sản phẩm cũng như cuốn báo cáo này.

Tôi cũng xin gửi lời cảm ơn đến quý Thầy, Cô trong Khoa **Công nghệ Thông tin** trường **Đại Học Trà Vinh** đã trang bị cho tôi nền tảng kiến thức vững chắc về lập trình hướng đối tượng, cơ sở dữ liệu và công nghệ phần mềm trong suốt thời gian học tập tại trường.

Mặc dù đã có nhiều cố gắng và nỗ lực trong quá trình phân tích, thiết kế và lập trình hệ thống, song do thời gian thực hiện cũng như kinh nghiệm thực tiễn của bản thân còn nhất định, đồ án khó tránh khỏi những thiếu sót. Tôi rất mong nhận được những ý kiến đóng góp, nhận xét quý báu từ quý Thầy, Cô để có thể tiếp tục hoàn thiện kiến thức và nâng cấp hệ thống tốt hơn trong tương lai.

Tôi xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN i
DANH MỤC CÁC CHỮ VIẾT TẮT vi
TÓM TẮT ĐỒ ÁN viii
MỞ ĐẦU 1
CHƯƠNG 1. TỔNG QUAN 5
1.1. Đặt vấn đề và bối cảnh thực tiễn 5
1.1.1. Xu hướng chuyển đổi số trong lĩnh vực lưu trú và khách sạn 5
1.1.2. Hiện trạng và những hạn chế của phương thức quản lý truyền thống 5
1.2. Giới thiệu tổng quan về hệ thống HotelManager 6
1.2.1. Định hướng xây dựng hệ thống 6
1.2.2. Giải pháp tiếp cận của đề tài 7
1.3. Phân tích các tác nhân và luồng nghiệp vụ tổng quát 8
1.3.1. Nhóm người dùng Quản trị viên (Administrator) 8
1.3.2. Nhóm người dùng Khách hàng (Client) 8
1.3.3. Nhóm khách vãng lai chưa định danh (Guest) 9
1.4. Các mục tiêu kỹ thuật và trải nghiệm người dùng cần đạt được 10
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT 11
2.1. Tổng quan về nền tảng .NET và ngôn ngữ lập trình C# 11
2.1.1. Nền tảng phát triển phần mềm .NET 11
2.1.2. Ngôn ngữ lập trình C# và lập trình hướng đối tượng 12
2.1.3. Ngôn ngữ truy vấn tích hợp (LINQ) và lập trình bất đồng bộ 13
2.2. Kiến trúc ứng dụng web ASP.NET Core MVC 14

2.2.1. Mô hình kiến trúc Model - View - Controller (MVC) 14
2.2.2. Chu trình xử lý yêu cầu (Request Pipeline) và Middleware 16
2.2.3. Cơ chế tiêm phụ thuộc (Dependency Injection - DI) 17
2.2.4. Công nghệ hiển thị Razor View Engine và Tag Helpers 18
2.3. Công nghệ truy cập dữ liệu và Hệ quản trị cơ sở dữ liệu 19
2.3.1. Kỹ thuật ánh xạ hướng đối tượng - quan hệ (ORM) và Entity Framework Core 19
2.3.2. Hướng tiếp cận Code-First và cơ chế Migrations 20
2.3.3. Hệ quản trị cơ sở dữ liệu Microsoft SQL Server (LocalDB) 21
2.4. Cơ chế xác thực và phân quyền với ASP.NET Core Identity 21
2.4.1. Khái niệm xác thực (Authentication) và phân quyền (Authorization) 21
2.4.2. Kiến trúc của ASP.NET Core Identity 22
2.4.3. Kiểm soát truy cập dựa trên vai trò (RBAC) và kỹ thuật Scaffolding 23
2.5. Công nghệ thiết kế giao diện và công cụ quản lý mã nguồn 24
2.5.1. Ngôn ngữ đánh dấu HTML5, CSS3 và bộ khung giao diện Bootstrap 5 24
2.5.2. Chủ đề giao diện Bootswatch Lux trong thiết kế trải nghiệm người dùng 25
2.5.3. Môi trường phát triển và công cụ quản lý phiên bản Git / GitHub Desktop 26
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU 27
3.1. Phân tích yêu cầu hệ thống 27
3.1.1. Xác định các tác nhân của hệ thống (Actors) 27
3.1.2. Yêu cầu chức năng (Functional Requirements) 29
3.1.3. Yêu cầu phi chức năng (Non-Functional Requirements) 31
3.2. Thiết kế hệ thống 32
3.2.1. Phân rã Use Case và đặc tả chi tiết 32

3.2.2. Đặc tả luồng hoạt động hệ thống (Activity Flows) 43
3.2.3. Thiết kế trình tự trao đổi thông điệp (Sequence Flow) 47
3.3. Thiết kế cơ sở dữ liệu 50
3.3.1. Đặc tả quan hệ thực thể (Entity Relationship - ERD) 51
3.3.2. Đặc tả chi tiết các bảng dữ liệu (Data Dictionary) 52
3.4. Hiện thực hóa và cài đặt chương trình 59
3.4.1. Tổ chức cấu trúc thư mục dự án ASP.NET Core MVC 59
3.4.2. Cài đặt tầng Model và Ngữ cảnh dữ liệu (`ApplicationDbContext`) 61
3.4.3. Hiện thực hóa cơ chế khởi tạo dữ liệu tự động (Data Seeding) trong `Program.cs` 67
3.4.4. Hiện thực hóa tầng Controller và bảo mật phân quyền (`[Authorize]`) 72
3.4.5. Hiện thực hóa tầng View và tinh chỉnh trải nghiệm người dùng (UI/UX) 79
3.4.6. Quản lý phiên bản và đồng bộ mã nguồn với GitHub Desktop 80
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU 82
4.1. Đánh giá kết quả giao diện người dùng (UI/UX) 82
4.1.1. Trang chủ và Danh mục phòng (Khách vãng lai và Khách hàng) 82
4.1.2. Phân hệ Xác thực và Kiểm soát tài khoản 83
4.1.3. Phân hệ Quản lý Đặt phòng (Dành cho Khách hàng) 84
4.1.4. Phân hệ Quản trị hệ thống (Dành cho Admin) 85
4.2. Kịch bản kiểm thử hệ thống (Test Cases) 86
4.3. Nhận xét kết quả đạt được 92
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 93
5.1. Kết luận 93

5.2. Hướng phát triển.....94
DANH MỤC TÀI LIỆU THAM KHẢO 96

DANH MỤC CÁC CHỮ VIẾT TẮT

(Đã được sắp xếp theo thứ tự bảng chữ cái chuẩn quy định)

Chữ viết tắt	**Tiếng Anh**	**Giải nghĩa tiếng Việt**
CSS	Cascading Style Sheets	Ngôn ngữ định dạng và trang trí giao diện trang web
CSDL	Database	Cơ sở dữ liệu
EF Core	Entity Framework Core	Khung ánh xạ hướng đối tượng và cơ sở dữ liệu cho nền tảng .NET
ERD	Entity Relationship Diagram	Sơ đồ mối quan hệ thực thể trong cơ sở dữ liệu
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
LINQ	Language Integrated Query	Ngôn ngữ truy vấn tích hợp trong C#

Chữ viết tắt	Tiếng Anh	Giải nghĩa tiếng Việt
MVC	Model - View - Controller	Mô hình kiến trúc phần mềm tách biệt Dữ liệu - Giao diện - Điều khiển
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ giữa đối tượng lập trình và cơ sở dữ liệu quan hệ
RBAC	Role-Based Access Control	Cơ chế kiểm soát truy cập và phân quyền dựa trên vai trò
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
UI	User Interface	Giao diện người dùng
URL	Uniform Resource Locator	Định vị tài nguyên thống nhất (Đường dẫn trang web)
UX	User Experience	Trải nghiệm người dùng

TÓM TẮT ĐỒ ÁN

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong ngành dịch vụ lưu trú và du lịch, việc tự động hóa quy trình quản lý danh mục phòng và tiếp nhận yêu cầu đặt phòng trực tuyến là yếu tố thiết yếu giúp các cơ sở kinh doanh khách sạn nâng cao hiệu quả vận hành và chất lượng phục vụ khách hàng. Đồ án này tập trung nghiên cứu và xây dựng hệ thống ứng dụng web **HotelManager** nhằm giải quyết bài toán quản lý thông tin phòng nghỉ, phân quyền người dùng và xử lý nghiệp vụ đặt phòng một cách trực quan, an toàn và đồng bộ.

Về hướng tiếp cận và công nghệ giải quyết vấn đề, hệ thống được thiết kế theo mô hình kiến trúc **Model - View - Controller (MVC)** trên nền tảng **ASP.NET Core** kết hợp với ngôn ngữ lập trình **C#**. Để tối ưu hóa quá trình tương tác với hệ quản trị cơ sở dữ liệu **Microsoft SQL Server (LocalDB)**, đồ án áp dụng kỹ thuật **Code-First** thông qua **Entity Framework Core**, cho phép khởi tạo cấu trúc bảng và quản lý lịch sử thay đổi dữ liệu tự động bằng cơ chế Migrations. Về mặt bảo mật, hệ thống tích hợp **ASP.NET Core Identity** kết hợp với cơ chế khởi tạo dữ liệu mẫu tự động (Data Seeding), giúp thiết lập và phân tách rõ ràng quyền hạn giữa Quản trị viên (Administrator) và Khách hàng (Client). Giao diện phía người dùng được thiết kế hiện đại, đồng bộ dựa trên bộ khung **Bootstrap 5** với giao diện tùy biến **Bootswatch Lux**, mang lại trải nghiệm sang trọng và phù hợp với đặc thù ngành khách sạn.

Kết quả đạt được của đồ án là một ứng dụng web hoàn chỉnh, vận hành ổn định với đầy đủ các luồng nghiệp vụ cốt lõi: hiển thị danh mục phòng trực quan trên trang chủ, xác thực và phân quyền truy cập chặt chẽ, cho phép khách hàng đặt phòng và tra cứu lịch sử đặt phòng cá nhân, đồng thời cung cấp công cụ để người quản lý thêm mới danh mục phòng và giám sát toàn bộ đơn đặt phòng trên hệ thống.

MỞ ĐẦU

1. Lý do chọn đề tài

Ngành dịch vụ lưu trú và khách sạn luôn là một trong những lĩnh vực có tính cạnh tranh cao, đòi hỏi sự chính xác trong khâu quản lý vận hành cũng như sự nhanh chóng, tiện lợi trong khâu giao tiếp với khách hàng. Trước đây, nhiều cơ sở lưu trú vừa và nhỏ thường quản lý danh sách phòng trống, thông tin khách hàng và lịch đặt phòng thông qua sổ sách giấy tờ hoặc các tệp bảng tính rời rạc. Phương thức quản lý truyền thống này bộc lộ nhiều hạn chế lớn như: tốn thời gian tra cứu, dễ xảy ra sai sót hoặc trùng lặp lịch đặt phòng, khó bảo mật dữ liệu và khách hàng không thể chủ động xem thông tin phòng hay đặt phòng từ xa.

Sự phát triển của công nghệ web hiện đại, đặc biệt là nền tảng ASP.NET Core do Microsoft phát triển, đã mang lại giải pháp mạnh mẽ để xây dựng các hệ thống thông tin quản lý trực tuyến có tính bảo mật cao, tốc độ xử lý nhanh và khả năng mở rộng linh hoạt. Xuất phát từ nhu cầu thực tiễn đó, cùng với mong muốn vận dụng các kiến thức đã học về lập trình C#, kiến trúc MVC và cơ sở dữ liệu quan hệ vào một sản phẩm hoàn chỉnh, tôi quyết định lựa chọn đề tài: **"Xây dựng ứng dụng web quản lý và đặt phòng khách sạn HotelManager bằng ASP.NET Core MVC"** làm đồ án môn học của mình.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đồ án là tìm hiểu sâu về quy trình phát triển một ứng dụng web toàn diện (Full-stack Web Application) trên nền tảng .NET và áp dụng vào việc xây dựng hệ thống quản lý đặt phòng khách sạn HotelManager. Cụ thể, đồ án hướng tới các mục tiêu sau:

- Nghiên cứu và nắm vững nguyên lý hoạt động của mô hình kiến trúc ASP.NET Core MVC, cách thức định tuyến (Routing), xử lý yêu cầu tại Controller và kết xuất giao diện qua Razor Views.
- Áp dụng thành thạo công cụ Entity Framework Core theo hướng tiếp cận Code-First để thiết kế mô hình dữ liệu, thiết lập mối quan hệ giữa các bảng và thao tác với cơ sở dữ liệu SQL Server thông qua LINQ.
- Tích hợp hệ thống bảo mật ASP.NET Core Identity nhằm giải quyết bài toán xác thực người dùng (Authentication) và phân quyền theo vai trò (Role-Based Authorization) giữa Quản lý và Khách hàng.
- Xây dựng giao diện người dùng (UI/UX) hiện đại, bố cục rõ ràng, dễ sử dụng và đảm bảo tính thẩm mỹ phù hợp với lĩnh vực dịch vụ khách sạn.

3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:**
    - Quy trình nghiệp vụ quản lý thông tin phòng nghỉ và quy trình đặt phòng trực tuyến tại các khách sạn quy mô vừa và nhỏ.
    - Nền tảng công nghệ ASP.NET Core MVC, ngôn ngữ lập trình C#, Entity Framework Core, hệ thống xác thực Microsoft Identity và thư viện giao diện Bootstrap 5.
- **Phạm vi nghiên cứu:**
    - Về mặt chức năng: Hệ thống tập trung giải quyết hai phân hệ người dùng chính là **Quản trị viên (Admin)** và **Khách hàng (Client)**. Bao gồm các chức năng: xem danh sách phòng, đặt phòng trực tuyến, theo dõi lịch sử đặt

phòng cá nhân, quản lý danh mục phòng (thêm mới, xem danh sách) và quản lý toàn bộ đơn đặt phòng của khách sạn.

- Về mặt kỹ thuật: Hệ thống được triển khai và kiểm thử trên môi trường máy chủ cục bộ kết nối với hệ quản trị cơ sở dữ liệu Microsoft SQL Server Express LocalDB, sử dụng cơ chế tự động khởi tạo tài khoản phân quyền mẫu và kiểm soát chặt chẽ việc đăng ký tài khoản trái phép.

4. Phương pháp nghiên cứu

Để hoàn thành các mục tiêu đề ra, đồ án kết hợp các phương pháp nghiên cứu sau:

- **Phương pháp nghiên cứu tài liệu:** Thu thập, đọc hiểu và tổng hợp kiến thức từ các tài liệu kỹ thuật chính thức của Microsoft (Microsoft Learn) về ASP.NET Core, Entity Framework Core, C# và các giáo trình công nghệ phần mềm.
- **Phương pháp phân tích và thiết kế hệ thống:** Khảo sát luồng nghiệp vụ thực tế, từ đó xác định các tác nhân (Actors), xây dựng sơ đồ Use Case, sơ đồ hoạt động (Activity Diagram) và thiết kế sơ đồ cơ sở dữ liệu quan hệ (ERD) trước khi tiến hành lập trình.
- **Phương pháp thực nghiệm và hoàn thiện lặp (Iterative Development):** Tiến hành cài đặt mã nguồn từng chức năng từ cơ sở dữ liệu, tầng xử lý nghiệp vụ đến giao diện hiển thị; thực hiện kiểm thử trực tiếp trên trình duyệt, phát hiện lỗi và tinh chỉnh giao diện người dùng liên tục cho đến khi đạt kết quả tối ưu.

5. Bố cục của Đồ án

Ngoài phần Mở đầu, Kết luận, Danh mục tài liệu tham khảo và Phụ lục, nội dung chính của đồ án được tổ chức thành 5 chương như sau:

- **Chương 1. Tổng quan:** Giới thiệu bối cảnh thực tiễn, bài toán quản lý đặt phòng khách sạn, các hạn chế của phương thức truyền thống và định hướng giải pháp của hệ thống HotelManager.
- **Chương 2. Nghiên cứu lý thuyết:** Trình bày cơ sở lý thuyết về ngôn ngữ C#, kiến trúc ASP.NET Core MVC, Entity Framework Core, SQL Server, ASP.NET Core Identity, Bootstrap và công cụ quản lý mã nguồn Git.
- **Chương 3. Hiện thực hóa nghiên cứu:** Mô tả chi tiết quá trình phân tích yêu cầu, thiết kế hệ thống qua các lược đồ UML, thiết kế cơ sở dữ liệu và giải thích cách thức cài đặt mã nguồn các chức năng cốt lõi.
- **Chương 4. Kết quả nghiên cứu:** Trình bày chi tiết các giao diện chức năng của sản phẩm sau khi hoàn thiện, kịch bản kiểm thử hệ thống và đánh giá trải nghiệm người dùng.
- **Chương 5. Kết luận và hướng phát triển:** Tổng kết những kết quả đã đạt được của đồ án, rút ra bài học kinh nghiệm và đề xuất các hướng nâng cấp, mở rộng hệ thống trong tương lai.

CHƯƠNG 1. TỔNG QUAN

1.1. Đặt vấn đề và bối cảnh thực tiễn

1.1.1. Xu hướng chuyển đổi số trong lĩnh vực lưu trú và khách sạn

Trong kỷ nguyên công nghệ thông tin phát triển mạnh mẽ, Internet và các ứng dụng web đã làm thay đổi hoàn toàn thói quen tìm kiếm thông tin cũng như hành vi tiêu dùng của khách hàng. Đối với ngành du lịch và dịch vụ lưu trú, thay vì phải đến trực tiếp quầy lễ tân hoặc gọi điện thoại để hỏi tình trạng phòng trống và giá cả như trước đây, khách hàng hiện đại ưu tiên thực hiện mọi thao tác tra cứu, so sánh và đặt phòng thông qua các nền tảng trực tuyến chỉ với vài thao tác trên máy tính hoặc thiết bị di động.

Về phía các đơn vị kinh doanh khách sạn, việc ứng dụng phần mềm quản lý trên nền tảng web không chỉ là giải pháp nâng cao hình ảnh chuyên nghiệp, sang trọng của thương hiệu mà còn là công cụ cốt lõi để tối ưu hóa quy trình vận hành nội bộ. Một hệ thống quản lý tập trung giúp kết nối trực tiếp nhu cầu của khách hàng với bộ phận quản lý khách sạn theo thời gian thực, giảm thiểu các khâu trung gian và nâng cao năng suất lao động.

1.1.2. Hiện trạng và những hạn chế của phương thức quản lý truyền thống

Qua khảo sát thực tế tại nhiều cơ sở lưu trú quy mô nhỏ hoặc các mô hình quản lý chưa áp dụng hệ thống phần mềm chuyên dụng, quy trình tiếp nhận và quản lý đặt phòng thường gặp phải những bất cập lớn sau:

1. **Thông tin phân tán và thiếu tính đồng bộ:** Danh mục phòng, mức giá, tình trạng phòng và danh sách khách đặt phòng thường được ghi chép trên sổ sách hoặc lưu trữ trong các tệp Excel riêng lẻ. Khi có sự thay đổi về giá phòng hoặc phát sinh đơn đặt phòng mới, dữ liệu không được cập nhật tức thời cho tất cả các bộ phận liên quan.

2. **Rủi ro sai sót trong quá trình đặt phòng:** Việc kiểm tra ngày nhận phòng (Check-in) và ngày trả phòng (Check-out) bằng mắt thường trên bảng tính rất dễ dẫn đến nhầm lẫn, ghi thiếu thông tin khách hàng hoặc thất lạc lịch đặt phòng vào các giai đoạn cao điểm.
3. **Thiếu tính chủ động cho khách hàng:** Khách hàng hoàn toàn phụ thuộc vào nhân viên trực tổng đài hoặc lễ tân để biết khách sạn có những loại phòng nào, mức giá bao nhiêu và đơn đặt phòng của mình đã được ghi nhận hay chưa. Khách hàng không có một không gian cá nhân trực tuyến để tự tra cứu lại lịch sử các phòng mình đã đặt.
4. **Hạn chế về bảo mật và phân quyền truy cập:** Khi lưu trữ dữ liệu trên các tệp văn bản thông thường, việc phân định ai là người có quyền chỉnh sửa cấu trúc giá phòng, ai chỉ có quyền xem thông tin là rất khó khăn. Bất kỳ sự thao tác nhầm nào cũng có thể làm mất mát hoặc sai lệch dữ liệu gốc của khách sạn.

Từ những hạn chế nêu trên, bài toán đặt ra là cần xây dựng một ứng dụng web quản lý khách sạn tập trung, có cơ sở dữ liệu chuẩn hóa, có giao diện trực quan cho khách hàng tự đặt phòng và tích hợp cơ chế phân quyền bảo mật chặt chẽ cho người quản lý.

1.2. Giới thiệu tổng quan về hệ thống HotelManager

1.2.1. Định hướng xây dựng hệ thống

Hệ thống **HotelManager** được xây dựng như một giải pháp phần mềm nền tảng web (Web-based Application) toàn diện, đóng vai trò là cầu nối trực tuyến giữa Ban quản lý khách sạn và Khách hàng có nhu cầu lưu trú. Hệ thống được định hướng theo ba tiêu chí cốt lõi:

- **Tính trực quan và sang trọng (UI/UX):** Giao diện trang web được thiết kế theo phong cách tối giản, hiện đại và cao cấp nhờ áp dụng chủ đề giao diện *Bootswatch*

Lux, kết hợp với hình ảnh đại diện lớn (Hero Banner) ngay tại trang chủ nhằm tạo ấn tượng thị giác chuyên nghiệp ngay khi người dùng truy cập.

- **Tính bảo mật và kiểm soát truy cập (Security & Access Control):** Mọi thao tác nhạy cảm như thêm phòng mới, xem toàn bộ danh sách đặt phòng của hệ thống hoặc thực hiện đặt phòng đều được kiểm soát bởi tầng xác thực danh tính người dùng. Hệ thống phân biệt rạch ròi quyền hạn giữa các nhóm tài khoản và ngăn chặn người dùng chưa được cấp phép truy cập trái phép qua đường dẫn URL.
- **Tính tự động hóa và sẵn sàng vận hành (Automation & Reliability):** Hệ thống tích hợp cơ chế tự động khởi tạo cấu trúc cơ sở dữ liệu và tài khoản quản trị/khách hàng mặc định ngay từ lần khởi chạy đầu tiên (Data Seeding), giúp quá trình triển khai phần mềm trên các môi trường máy tính khác nhau diễn ra nhanh chóng mà không cần can thiệp cấu hình thủ công vào cơ sở dữ liệu.

1.2.2. Giải pháp tiếp cận của đề tài

Để hiện thực hóa hệ thống HotelManager, đồ án lựa chọn hướng tiếp cận phát triển phần mềm theo kiến trúc phân tầng **ASP.NET Core MVC**. Trong kiến trúc này:

- Tầng **Model** phối hợp cùng **Entity Framework Core** chịu trách nhiệm định nghĩa cấu trúc các thực thể nghiệp vụ (như Phòng nghỉ `Room`, Đơn đặt phòng `Booking`, Người dùng `IdentityUser`) và ánh xạ trực tiếp xuống các bảng trong hệ quản trị cơ sở dữ liệu SQL Server.
- Tầng **Controller** đóng vai trò là bộ não điều phối, tiếp nhận các yêu cầu HTTP từ trình duyệt của người dùng, kiểm tra quyền hạn thông qua các thuộc tính bảo mật (`[Authorize]`), thực thi các truy vấn dữ liệu và trả về kết quả tương ứng.
- Tầng **View** sử dụng công nghệ **Razor Engine** kết hợp với HTML5 và CSS3/Bootstrap 5 để hiển thị dữ liệu một cách linh hoạt. Thanh điều hướng (Navigation Bar) và các nút chức năng trên giao diện sẽ tự động thay đổi dựa trên trạng thái đăng nhập và vai trò của người dùng hiện tại.

1.3. Phân tích các tác nhân và luồng nghiệp vụ tổng quát

Hệ thống HotelManager được thiết kế xoay quanh sự tương tác của 3 nhóm tác nhân (Actors) chính, mỗi nhóm có phạm vi quyền hạn và luồng nghiệp vụ riêng biệt:

1.3.1. Nhóm người dùng Quản trị viên (Administrator)

Quản trị viên (sử dụng tài khoản quản lý hệ thống) là người nắm giữ quyền hạn cao nhất trong việc điều hành hoạt động của khách sạn trên phần mềm. Các nghiệp vụ chính của Quản trị viên bao gồm:

- **Đăng nhập hệ thống quản trị:** Xác thực danh tính thông qua trang đăng nhập bảo mật để kích hoạt các công cụ dành riêng cho người quản lý.
- **Quản lý danh mục phòng (Room Management):** Xem toàn bộ danh sách các phòng hiện có trong khách sạn kèm theo thông tin chi tiết (số phòng/tên phòng, loại phòng, giá tiền mỗi đêm, tình trạng phòng). Quản trị viên có quyền truy cập biểu mẫu thêm mới phòng nghỉ để bổ sung vào cơ sở dữ liệu khi khách sạn mở rộng quy mô.
- **Quản lý toàn bộ đơn đặt phòng (All Bookings Management):** Theo dõi và kiểm soát danh sách tất cả các đơn đặt phòng do mọi khách hàng gửi lên hệ thống, bao gồm thông tin email khách đặt, phòng được đặt, ngày nhận phòng và ngày trả phòng nhằm phục vụ công tác điều phối lễ tân.

1.3.2. Nhóm người dùng Khách hàng (Client)

Khách hàng là những người dùng đã được cấp tài khoản hợp lệ trên hệ thống để sử dụng dịch vụ đặt phòng trực tuyến. Các nghiệp vụ chính của Khách hàng bao gồm:

- **Khám phá và tra cứu thông tin phòng:** Xem giao diện trang chủ giới thiệu khách sạn và duyệt qua danh mục các phòng nghỉ đang mở bán với thông tin giá cả minh bạch.
- **Thực hiện đặt phòng trực tuyến (Book a Room):** Lựa chọn phòng nghỉ ưng ý, nhập thông tin thời gian lưu trú dự kiến (ngày nhận phòng và ngày trả phòng) để tạo đơn đặt phòng mới trên hệ thống. Hệ thống sẽ tự động gắn kết đơn đặt phòng này với tài khoản đang đăng nhập của khách hàng.
- **Quản lý đơn đặt phòng cá nhân (My Bookings):** Truy cập vào trang quản lý cá nhân để xem lại danh sách các phòng mà chính bản thân khách hàng đó đã đặt. Hệ thống đảm bảo tính riêng tư tuyệt đối: một khách hàng chỉ có thể nhìn thấy lịch sử đặt phòng của chính mình và không thể xem được đơn đặt phòng của khách hàng khác.

1.3.3. Nhóm khách vãng lai chưa định danh (Guest)

Khách vãng lai là những người dùng truy cập vào trang web nhưng chưa thực hiện đăng nhập:

- Được phép xem trang chủ (Home), trang chính sách bảo mật (Privacy) và xem danh sách các phòng hiện có của khách sạn để tham khảo thông tin.
- Khi khách vãng lai có nhu cầu bấm đặt phòng hoặc truy cập vào các đường dẫn quản lý, hệ thống sẽ tự động chặn yêu cầu và chuyển hướng người dùng về trang Đăng nhập (Login).
- Đặc biệt, nhằm đảm bảo an toàn cho hệ thống nội bộ và tránh việc tạo tài khoản rác tự do trong giai đoạn vận hành kiểm soát, khi người dùng bấm vào mục Đăng ký tài khoản mới (Register), hệ thống sẽ điều hướng sang trang thông báo tính năng đang được bảo trì/xây dựng (*Under Construction*) một cách thân thiện và chuyên nghiệp.

1.4. Các mục tiêu kỹ thuật và trải nghiệm người dùng cần đạt được

Để đánh giá mức độ hoàn thiện của hệ thống HotelManager, đồ án đặt ra các tiêu chuẩn kỹ thuật và trải nghiệm người dùng cụ thể như sau:

1. **Tính toàn vẹn dữ liệu:** Mọi đơn đặt phòng (`Booking`) khi lưu vào cơ sở dữ liệu phải liên kết chính xác với mã phòng (`RoomId`) và thông tin định danh của người đặt (`UserId` / `UserEmail`), không xảy ra tình trạng dữ liệu mồ côi hoặc sai lệch kiểu dữ liệu.
2. **Bảo mật hai lớp (Giao diện và Máy chủ):** Việc phân quyền không chỉ dừng lại ở việc ẩn các nút chức năng của Admin đối với tài khoản Client trên giao diện, mà còn phải được khóa chặt tại tầng Controller ở phía máy chủ. Nếu một người dùng thông thường cố tình nhập trực tiếp đường dẫn tạo phòng mới lên thanh địa chỉ trình duyệt, hệ thống phải từ chối quyền truy cập ngay lập tức.
3. **Giao diện tinh gọn và nhất quán:** Loại bỏ các thành phần giao diện thừa hoặc không sử dụng của bộ khung mặc định (chẳng hạn như khu vực đăng nhập bằng dịch vụ bên thứ ba không cần thiết ở trang Login), đồng thời căn chỉnh bố cục cân đối, chữ viết rõ ràng, độ tương phản tốt trên mọi trang chức năng.

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

2.1. Tổng quan về nền tảng .NET và ngôn ngữ lập trình C#

2.1.1. Nền tảng phát triển phần mềm .NET

.NET (trước đây được biết đến với tên gọi .NET Core) là một nền tảng phát triển phần mềm mã nguồn mở, đa nền tảng (Cross-platform) và hiệu năng cao do tập đoàn Microsoft phát triển và duy trì. Khác với thế hệ .NET Framework truyền thống vốn chỉ hoạt động giới hạn trên hệ điều hành Windows, nền tảng .NET hiện đại cho phép các nhà phát triển xây dựng và triển khai ứng dụng linh hoạt trên nhiều hệ điều hành khác nhau như Windows, macOS và các bản phân phối của Linux.

Một hệ sinh thái .NET hoàn chỉnh được cấu thành từ ba thành phần kỹ thuật cốt lõi:

- **Môi trường thực thi thời gian chạy (Common Language Runtime - CLR):** Đóng vai trò là tầng quản lý việc thực thi mã nguồn. CLR chịu trách nhiệm biên dịch mã trung gian (Intermediate Language - IL) sang mã máy gốc thông qua trình biên dịch Just-In-Time (JIT), đồng thời quản lý cấp phát bộ nhớ, thu gom rác tự động (Garbage Collection), xử lý ngoại lệ (Exception Handling) và đảm bảo an toàn kiểu dữ liệu.
- **Thư viện lớp cơ sở (Base Class Library - BCL):** Cung cấp một tập hợp phong phú các lớp, giao diện (Interfaces) và kiểu dữ liệu dựng sẵn, hỗ trợ lập trình viên xử lý các tác vụ nền tảng như thao tác chuỗi, quản lý tập hợp (Collections), đọc/ghi tệp tin, kết nối mạng và truy cập cơ sở dữ liệu.
- **Bộ công cụ phát triển phần mềm (.NET SDK và CLI):** Cung cấp trình biên dịch cùng giao diện dòng lệnh (`dotnet`) giúp tự động hóa việc khởi tạo dự án, quản lý

các gói thư viện bên thứ ba thông qua trình quản lý gói NuGet, biên dịch mã nguồn và xuất bản ứng dụng.

Trong đồ án xây dựng hệ thống **HotelManager**, nền tảng .NET đóng vai trò là bệ phóng vững chắc giúp ứng dụng đạt được tốc độ phản hồi nhanh, quản lý tài nguyên máy chủ tối ưu và dễ dàng tích hợp các thư viện bảo mật cũng như cơ sở dữ liệu đồng bộ của Microsoft.

2.1.2. Ngôn ngữ lập trình C# và lập trình hướng đối tượng

C# (đọc là *C-Sharp*) là ngôn ngữ lập trình hiện đại, định kiểu mạnh (Strongly-typed) và thuần hướng đối tượng được thiết kế chuyên biệt cho nền tảng .NET. Cú pháp của C# kế thừa sự chặt chẽ của họ ngôn ngữ C/C++ kết hợp với tính trực quan, an toàn và khả năng phát triển nhanh.

Trong quá trình xây dựng ứng dụng **HotelManager**, toàn bộ cấu trúc dữ liệu và luồng xử lý nghiệp vụ đều vận dụng bốn nguyên lý cơ bản của Lập trình hướng đối tượng (Object-Oriented Programming - OOP) trong C#:

1. **Tính đóng gói (Encapsulation):** Dữ liệu và các phương thức thao tác trên dữ liệu đó được gom chung vào trong một đơn vị duy nhất gọi là Lớp (Class). Thông qua các từ khóa phạm vi truy cập (`public`, `private`, `protected`) và cơ chế Thuộc tính (Properties với hai phương thức `get`/`set`), hệ thống bảo vệ trạng thái nội tại của các đối tượng nghiệp vụ như Phòng nghỉ (`Room`) hay Đơn đặt phòng (`Booking`), đảm bảo dữ liệu không bị thay đổi bất hợp lệ từ bên ngoài.
2. **Tính kế thừa (Inheritance):** Cho phép một lớp con kế thừa lại các thuộc tính và phương thức đã được định nghĩa sẵn từ một lớp cha, giúp tái sử dụng mã nguồn và giảm thiểu sự trùng lặp. Trong dự án này, tính kế thừa thể hiện rõ nét khi các bộ

điều khiển của ứng dụng (như `HomeController`, `RoomsController`, `BookingsController`) kế thừa từ lớp cơ sở `Controller` của ASP.NET Core, hoặc lớp ngữ cảnh dữ liệu `ApplicationDbContext` kế thừa từ lớp `IdentityDbContext` để thừa hưởng toàn bộ cấu trúc bảng bảo mật người dùng.

3. **Tính đa hình (Polymorphism):** Cho phép các đối tượng thuộc các lớp khác nhau có thể phản hồi theo những cách riêng biệt đối với cùng một thông điệp hoặc lời gọi phương thức, đồng thời hỗ trợ việc nạp chồng phương thức (Method Overloading) và ghi đè phương thức (Method Overriding). Ví dụ, trong một Controller, hai phương thức cùng mang tên `Create()` nhưng có danh sách tham số khác nhau sẽ đảm nhận hai vai trò riêng biệt: một phương thức xử lý yêu cầu `HTTP GET` để hiển thị biểu mẫu trống, và một phương thức xử lý yêu cầu `HTTP POST` để tiếp nhận và lưu dữ liệu người dùng gửi lên.
4. **Tính trừu tượng (Abstraction):** Tập trung vào việc mô hình hóa các đặc điểm cốt lõi của đối tượng trong thế giới thực vào phần mềm và ẩn đi các chi tiết cài đặt phức tạp bên dưới thông qua các lớp trừu tượng (Abstract Classes) và giao diện (Interfaces).

2.1.3. Ngôn ngữ truy vấn tích hợp (LINQ) và lập trình bất đồng bộ

Bên cạnh các đặc tính hướng đối tượng, C# cung cấp hai tính năng kỹ thuật quan trọng được sử dụng xuyên suốt trong hệ thống **HotelManager**:

- **Ngôn ngữ truy vấn tích hợp (Language Integrated Query - LINQ):** LINQ cho phép lập trình viên viết các câu lệnh truy vấn, lọc, sắp xếp và gom nhóm dữ liệu trực tiếp bằng cú pháp C# ngay trong mã nguồn thay vì phải viết các chuỗi câu lệnh SQL thuần túy. Thông qua các biểu thức Lambda (ví dụ: `.Where(b => b.UserId == currentUserId)` hoặc `.Include(b => b.Room)`), trình

biên dịch C# sẽ kiểm tra lỗi cú pháp và kiểu dữ liệu ngay tại thời điểm viết mã (Compile-time checking), sau đó tự động dịch sang ngôn ngữ SQL tương ứng để thực thi dưới cơ sở dữ liệu.

- **Lập trình bất đồng bộ với cặp từ khóa `async` và `await`:** Trong các ứng dụng web, các thao tác đọc/ghi cơ sở dữ liệu hoặc xác thực tài khoản người dùng thường tốn thời gian chờ phản hồi từ ổ cứng hoặc mạng (I/O-bound operations). Việc áp dụng cơ chế bất đồng bộ (`async Task<IActionResult>`) giúp giải phóng luồng xử lý (Thread) của máy chủ web trong lúc chờ cơ sở dữ liệu trả kết quả, giúp hệ thống không bị treo hoặc nghên cổ chai khi có nhiều khách hàng truy cập và đặt phòng cùng một lúc.

2.2. Kiến trúc ứng dụng web ASP.NET Core MVC

2.2.1. Mô hình kiến trúc Model - View - Controller (MVC)

ASP.NET Core MVC là một bộ khung phát triển ứng dụng web (Web Framework) triển khai theo mẫu thiết kế kiến trúc **Model - View - Controller**. Mục tiêu cốt lõi của kiến trúc MVC là nguyên lý **Tách biệt các mối quan tâm (Separation of Concerns - SoC)**, nghĩa là chia nhỏ một ứng dụng phức tạp thành ba thành phần độc lập nhưng phối hợp chặt chẽ với nhau:

1. **Tầng dữ liệu và nghiệp vụ (Model):**

    - Đại diện cho cấu trúc dữ liệu của ứng dụng và các quy tắc nghiệp vụ cốt lõi (Business Logic).

- Trong hệ thống **HotelManager**, tầng Model bao gồm các lớp thực thể C# như `Room` (chứa thông tin mã phòng, tên/số phòng, giá tiền, trạng thái) và `Booking` (chứa thông tin mã đặt phòng, ngày nhận phòng, ngày trả phòng, thông tin khách hàng và khóa ngoại liên kết tới phòng).
- Tầng Model còn sử dụng các thuộc tính chú thích dữ liệu (Data Annotations như `[Required]`, `[Display]`, `[DataType]`) để kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi lưu xuống cơ sở dữ liệu.

2. **Tầng giao diện hiển thị (View):**

- Đảm nhận nhiệm vụ trình bày dữ liệu lên màn hình trình duyệt cho người dùng cuối thông qua HTML, CSS và JavaScript.
- Tầng View hoàn toàn bị động: nó không trực tiếp truy vấn cơ sở dữ liệu hay xử lý nghiệp vụ phức tạp, mà chỉ nhận dữ liệu đã được chuẩn bị sẵn từ Controller để hiển thị thành các bảng danh sách phòng, biểu mẫu đặt phòng hoặc trang đăng nhập.

3. **Tầng điều khiển trung tâm (Controller):**

- Đóng vai trò là cầu nối điều phối giữa người dùng, tầng Model và tầng View.
- Khi người dùng nhấp chuột vào một liên kết hoặc bấm nút gửi biểu mẫu trên trình duyệt, yêu cầu HTTP sẽ được chuyển đến một phương thức hành động (Action Method) tương ứng bên trong Controller. Controller sẽ kiểm tra quyền hạn của người dùng, gọi tầng Model/Cơ sở dữ liệu để lấy hoặc

cập nhật thông tin, sau đó quyết định chọn View nào để trả về cho trình duyệt hiển thị.

Việc áp dụng kiến trúc MVC mang lại lợi ích to lớn cho quá trình phát triển và bảo trì hệ thống: khi cần thay đổi giao diện trang web (chẳng hạn đổi sang chủ đề giao diện *Bootswatch Lux* hoặc căn giữa lại trang Đăng nhập), lập trình viên chỉ cần chỉnh sửa ở tầng View mà không làm ảnh hưởng hay gây lỗi cho các thuật toán xử lý đặt phòng ở tầng Controller và Model.

2.2.2. Chu trình xử lý yêu cầu (Request Pipeline) và Middleware

Trong một ứng dụng ASP.NET Core, mọi yêu cầu HTTP gửi từ trình duyệt của khách hàng đến máy chủ và mọi phản hồi trả ngược về đều phải đi qua một đường ống xử lý tuần tự được gọi là **Request Pipeline**. Đường ống này được lắp ghép từ một chuỗi các thành phần phần mềm trung gian gọi là **Middleware** (được cấu hình trong tệp `Program.cs`).

Mỗi Middleware thực hiện một nhiệm vụ chuyên biệt trên yêu cầu HTTP trước khi chuyển giao yêu cầu đó cho Middleware kế tiếp trong chuỗi:

- `UseHttpsRedirection()`: Tự động chuyển hướng các kết nối HTTP thông thường sang giao thức bảo mật HTTPS.
- `UseStaticFiles()`: Cho phép máy chủ trả về ngay lập tức các tệp tài nguyên tĩnh nằm trong thư mục `wwwroot` như hình ảnh (ảnh nền Hero Banner, ảnh phòng), tệp định dạng CSS và tệp kịch bản JavaScript mà không cần đi qua tầng xử lý phức tạp.
- `UseRouting()`: Phân tích đường dẫn URL mà người dùng yêu cầu để xác định xem yêu cầu đó muốn gọi đến Controller và Action nào.

- `UseAuthentication()`: Kiểm tra thông tin định danh (Cookie hoặc Token) đính kèm trong yêu cầu để xác định người dùng hiện tại là ai (đã đăng nhập hay chưa).
- `UseAuthorization()`: Kiểm tra xem người dùng đã được xác thực đó có đủ quyền hạn (Role) để truy cập vào tài nguyên đang yêu cầu hay không. Thứ tự đặt `UseAuthentication()` trước `UseAuthorization()` là quy tắc bắt buộc trong ASP.NET Core.
- `MapControllerRoute()` và `MapRazorPages()`: Điểm cuối của đường ống (Endpoints), nơi thực thi phương thức trong Controller hoặc trang Razor tương ứng và tạo ra mã HTML trả về cho người dùng.

2.2.3. Cơ chế tiêm phụ thuộc (Dependency Injection - DI)

ASP.NET Core tích hợp sẵn nguyên lý **Tiêm phụ thuộc (Dependency Injection)** ngay từ nhân hệ thống mà không cần cài đặt thêm thư viện bên ngoài. Thay vì để một Controller tự khởi tạo cứng các đối tượng dịch vụ mà nó cần dùng (thông qua từ khóa `new`), các dịch vụ này sẽ được đăng ký tập trung tại vùng chứa dịch vụ (`builder.Services` trong tệp `Program.cs`).

Khi một yêu cầu gửi đến Controller, hệ thống sẽ tự động khởi tạo và truyền ("tiêm") các dịch vụ cần thiết như ngữ cảnh cơ sở dữ liệu (`ApplicationDbContext`) hoặc trình quản lý người dùng (`UserManager<IdentityUser>`) vào thông qua hàm khởi tạo (Constructor) của Controller. Cơ chế này giúp mã nguồn trở nên lỏng lẻo về mặt phụ thuộc (Loose Coupling), quản lý vòng đời bộ nhớ của kết nối cơ sở dữ liệu an toàn theo từng yêu cầu HTTP (Scoped Lifetime) và giúp mã nguồn gọn gàng, dễ kiểm soát.

2.2.4. Công nghệ hiển thị Razor View Engine và Tag Helpers

Để xây dựng tầng View động, ASP.NET Core sử dụng công cụ kết xuất giao diện **Razor View Engine** với các tệp có phần mở rộng là `.cshtml` (C# HTML). Công nghệ Razor cho phép nhúng trực tiếp các đoạn mã C# vào bên trong cấu trúc HTML thông qua ký tự `@`. Trước khi gửi trang web về cho trình duyệt của khách hàng, máy chủ sẽ thực thi toàn bộ các khối mã C# này (như vòng lặp `@foreach` duyệt qua danh sách phòng, hoặc cấu trúc rẽ nhánh `@if` kiểm tra trạng thái đăng nhập) để sinh ra văn bản HTML thuần túy.

Hệ thống giao diện Razor được tổ chức khoa học thông qua ba thành phần chính:

- **Trang bố cục dùng chung (`_Layout.cshtml`):** Đóng vai trò là khung xương sống (Master Page) của toàn bộ trang web, định nghĩa thanh điều hướng (Navigation Bar) ở phía trên, chân trang (Footer) ở phía dưới và các liên kết tới thư viện CSS/Bootstrap. Nội dung riêng của từng trang chức năng sẽ được chèn động vào vị trí của phương thức `@RenderBody()`.
- **Trang thành phần cục bộ (`_LoginPartial.cshtml`):** Là một khối giao diện nhỏ được nhúng vào thanh điều hướng của `_Layout.cshtml`, chuyên trách việc hiển thị nút *Login / Register* khi khách chưa đăng nhập, hoặc hiển thị *Xin chào [Email]* kèm nút *Logout* khi người dùng đã đăng nhập thành công.
- **Trợ giúp thẻ (Tag Helpers):** Là tính năng cho phép mã C# phía máy chủ tham gia vào việc tạo và điều khiển các thẻ HTML một cách tự nhiên. Ví dụ, thay vì phải viết các đường dẫn URL thủ công dễ sai sót, lập trình viên sử dụng các thuộc tính như `asp-controller="Bookings"`, `asp-action="MyBookings"` hoặc `asp-for="RoomId"`, hệ thống sẽ tự động sinh ra đường dẫn chuẩn xác và gắn kèm mã chống giả mạo yêu cầu (Anti-forgery Token) cho các biểu mẫu nhập liệu.

2.3. Công nghệ truy cập dữ liệu và Hệ quản trị cơ sở dữ liệu

2.3.1. Kỹ thuật ánh xạ hướng đối tượng - quan hệ (ORM) và Entity Framework Core

Trong phát triển phần mềm truyền thống, việc kết nối giữa ngôn ngữ lập trình hướng đối tượng (như C#) và hệ quản trị cơ sở dữ liệu quan hệ (như SQL Server) luôn tồn tại một sự khác biệt về mặt cấu trúc: trong khi C# quản lý dữ liệu dưới dạng các Đối tượng (Objects) có thuộc tính và tham chiếu phân cấp, thì SQL Server lại lưu trữ dữ liệu dưới dạng các Bảng (Tables) gồm các dòng và cột phẳng.

Kỹ thuật ánh xạ hướng đối tượng - quan hệ (Object-Relational Mapping - ORM) ra đời nhằm giải quyết sự bất tương thích này, và **Entity Framework Core (EF Core)** chính là bộ khung ORM chính thống, hiện đại và mạnh mẽ nhất của Microsoft dành cho nền tảng .NET.

Trong dự án **HotelManager**, EF Core hoạt động thông qua hai khái niệm trung tâm:

- **Lớp ngữ cảnh dữ liệu (`DbContext`):** Đại diện cho một phiên làm việc với cơ sở dữ liệu. Lớp `ApplicationDbContext` của dự án đóng vai trò là cầu nối quản lý chuỗi kết nối (Connection String), theo dõi trạng thái thay đổi của các đối tượng (Change Tracking) và thực thi việc lưu trữ dữ liệu xuống SQL Server thông qua phương thức `SaveChanges()` / `SaveChangesAsync()`.
- **Tập thực thể (`DbSet<TEntity>`):** Mỗi thuộc tính kiểu `DbSet<T>` được khai báo bên trong `ApplicationDbContext` (như `DbSet<Room> Rooms` và `DbSet<Booking> Bookings`) sẽ đại diện trực tiếp cho một bảng vật lý tương

ứng trong cơ sở dữ liệu. Mọi thao tác thêm (`Add`), sửa (`Update`), xóa (`Remove`) hay truy vấn trên `DbSet` đều được EF Core tự động dịch thành các câu lệnh SQL chuẩn xác (`INSERT`, `UPDATE`, `DELETE`, `SELECT`).

2.3.2. Hướng tiếp cận Code-First và cơ chế Migrations

Entity Framework Core hỗ trợ hai phương pháp tiếp cận chính khi thiết kế cơ sở dữ liệu là *Database-First* (thiết kế bảng trong SQL rồi sinh ra mã C#) và *Code-First* (viết các lớp C# rồi tự động sinh ra cơ sở dữ liệu). Đồ án **HotelManager** lựa chọn áp dụng hoàn toàn phương pháp **Code-First** vì những ưu điểm vượt trội trong quy trình phát triển phần mềm hiện đại:

1. **Lấy mã nguồn làm trung tâm:** Lập trình viên tập trung hoàn toàn vào việc thiết kế các lớp mô hình nghiệp vụ bằng ngôn ngữ C# (`Room.cs`, `Booking.cs`). EF Core sẽ tự động đọc cấu trúc các lớp này, nhận diện khóa chính (`Id`), khóa ngoại (`RoomId`) và kiểu dữ liệu để xây dựng sơ đồ cơ sở dữ liệu.
2. **Quản lý lịch sử phiên bản cơ sở dữ liệu với Migrations:** Khi yêu cầu bài toán thay đổi (chẳng hạn cần thêm một cột mới vào bảng `Bookings` hoặc thêm bảng mới), thay vì phải mở công cụ quản lý cơ sở dữ liệu để sửa bảng thủ công, lập trình viên chỉ cần cập nhật lớp C# và chạy lệnh tạo **Migration**. Mỗi bản Migration là một tệp mã C# ghi lại chính xác các bước nâng cấp (`Up`) hoặc hoàn tác (`Down`) cấu trúc cơ sở dữ liệu theo từng mốc thời gian.
3. **Tính di động và đồng bộ cao:** Khi đưa mã nguồn dự án lên kho lưu trữ trực tuyến và tải về một máy tính khác, hệ thống chỉ cần thực thi các bản Migrations có sẵn trong dự án là toàn bộ cấu trúc bảng sẽ được tái tạo giống hệt 100% mà không cần sao chép các tệp cơ sở dữ liệu dung lượng lớn.

2.3.3. Hệ quản trị cơ sở dữ liệu Microsoft SQL Server (LocalDB)

Để lưu trữ dữ liệu bền vững, hệ thống sử dụng **Microsoft SQL Server**, cụ thể là phiên bản **SQL Server Express LocalDB**. Đây là một phiên bản thu gọn của hệ quản trị cơ sở dữ liệu quan hệ SQL Server được thiết kế chuyên biệt cho các nhà phát triển phần mềm.

Ưu điểm lớn của SQL Server LocalDB là nó chạy dưới dạng một tiến trình nền nhẹ nhàng theo yêu cầu của ứng dụng (On-demand process) ngay dưới quyền của tài khoản người dùng Windows hiện tại, không tiêu tốn bộ nhớ hệ thống khi ứng dụng không hoạt động, nhưng vẫn hỗ trợ đầy đủ các tính năng cốt lõi của hệ quản trị cơ sở dữ liệu SQL Server tiêu chuẩn như: giao dịch an toàn (Transactions - ACID), ràng buộc khóa chính - khóa ngoại (Primary Key / Foreign Key Constraints) và ngôn ngữ truy vấn T-SQL. Khi ứng dụng cần triển khai lên máy chủ thực tế (Production), lập trình viên chỉ cần thay đổi chuỗi kết nối trong tệp cấu hình `appsettings.json` sang máy chủ SQL Server Enterprise hoặc Azure SQL Database mà không cần phải viết lại bất kỳ dòng mã xử lý dữ liệu nào.

2.4. Cơ chế xác thực và phân quyền với ASP.NET Core Identity

2.4.1. Khái niệm xác thực (Authentication) và phân quyền (Authorization)

Trong bất kỳ hệ thống thông tin quản lý nào có sự tham gia của nhiều nhóm đối tượng khác nhau, bảo mật ứng dụng luôn dựa trên hai trụ cột tách biệt nhưng bổ trợ chặt chẽ cho nhau:

- **Xác thực danh tính (Authentication):** Là quá trình trả lời cho câu hỏi *"Bạn là ai?"*. Quá trình này diễn ra khi người dùng cung cấp thông tin đăng nhập (địa chỉ Email và Mật khẩu) tại trang Login. Hệ thống sẽ đối chiếu thông tin này với dữ

liệu đã được mã hóa băm (Hashed Password) trong cơ sở dữ liệu. Nếu trùng khớp, hệ thống sẽ cấp một phiếu định danh điện tử (Authentication Cookie) lưu trên trình duyệt để ghi nhận trạng thái đã đăng nhập của người dùng đó trong các lần chuyển trang tiếp theo.

- **Phân quyền truy cập (Authorization):** Là quá trình diễn ra sau khi đã xác thực thành công, nhằm trả lời cho câu hỏi *"Bạn được phép làm những gì?"*. Dù hai người dùng đều đăng nhập hợp lệ vào hệ thống **HotelManager**, nhưng cơ chế phân quyền sẽ quyết định ai được phép vào trang thêm phòng mới của khách sạn (Quản trị viên) và ai chỉ được phép đặt phòng cũng như xem lịch sử cá nhân của mình (Khách hàng).

2.4.2. Kiến trúc của ASP.NET Core Identity

Thay vì tự xây dựng cơ chế đăng nhập thủ công vốn tiềm ẩn nhiều lỗ hổng bảo mật (như lưu mật khẩu dạng văn bản thuần hoặc dễ bị tấn công giả mạo phiên làm việc), đồ án tích hợp bộ khung **ASP.NET Core Identity** chính chủ của Microsoft. Hệ thống này cung cấp một kiến trúc quản lý người dùng tiêu chuẩn công nghiệp bao gồm:

- **Hệ thống bảng cơ sở dữ liệu tự động:** Khi tích hợp Identity, EF Core tự động khởi tạo một hệ thống các bảng chuyên dụng có tiền tố `AspNet...` trong cơ sở dữ liệu, quan trọng nhất là:
    - `AspNetUsers`: Lưu trữ thông tin tài khoản người dùng (ID dạng chuỗi GUID, UserName, Email, mật khẩu đã mã hóa băm `PasswordHash`, số điện thoại, trạng thái khóa tài khoản).
    - `AspNetRoles`: Lưu trữ danh sách các vai trò/nhóm quyền trong hệ thống (ví dụ: vai trò `Admin` và vai trò `Client`).
    - `AspNetUserRoles`: Bảng trung gian biểu diễn mối quan hệ nhiều - nhiều, xác định tài khoản người dùng nào đang nắm giữ vai trò nào.

- **Các lớp dịch vụ quản lý (Identity Managers):**
    - `UserManager<IdentityUser>`: Cung cấp các phương thức API để tạo mới người dùng, tìm kiếm người dùng theo Email/ID, kiểm tra mật khẩu và gán vai trò cho người dùng.
    - `RoleManager<IdentityRole>`: Cung cấp các phương thức kiểm tra sự tồn tại của một vai trò và khởi tạo các vai trò mới trong hệ thống.
    - `SignInManager<IdentityUser>`: Chịu trách nhiệm xử lý luồng đăng nhập bằng mật khẩu (`PasswordSignInAsync`), phát hành Cookie phiên làm việc và xử lý đăng xuất (`SignOutAsync`).

2.4.3. Kiểm soát truy cập dựa trên vai trò (RBAC) và kỹ thuật Scaffolding

Hệ thống **HotelManager** áp dụng mô hình **Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC)** kết hợp với cơ chế khởi tạo dữ liệu gốc tự động (**Data Seeding**):

1. **Khởi tạo dữ liệu phân quyền tự động (Data Seeding):** Ngay khi ứng dụng khởi động tại tệp `Program.cs`, hệ thống tự động kiểm tra cơ sở dữ liệu; nếu chưa có các vai trò `Admin` và `Client`, hệ thống sẽ tự động tạo mới chúng, đồng thời khởi tạo sẵn tài khoản Quản lý (`boss@hotel.com`) và tài khoản Khách hàng (`client@hotel.com`) rồi gán vào đúng vai trò tương ứng.
2. **Bảo vệ tầng Controller bằng thuộc tính `[Authorize]`:** Để ngăn chặn việc truy cập trái phép, các phương thức hoặc toàn bộ lớp Controller được gắn nhãn `[Authorize]`. Khi sử dụng cú pháp `[Authorize(Roles = "Admin")]`, chỉ những tài khoản có vai trò `Admin` mới được phép thực thi phương thức đó; mọi yêu cầu từ tài khoản `Client` hoặc người dùng chưa đăng nhập đều sẽ bị máy chủ chặn lại ngay lập tức.

3. **Kỹ thuật trích xuất và tùy biến giao diện Identity (Identity Scaffolding):** Mặc định, các trang giao diện của ASP.NET Core Identity (như trang Đăng nhập `Login` và trang Đăng ký `Register`) được đóng gói ẩn bên trong thư viện lớp biên dịch sẵn (Razor Class Library). Thông qua công cụ `Microsoft.VisualStudio.Web.CodeGeneration.Design`, lập trình viên thực hiện kỹ thuật **Scaffolding** để trích xuất mã nguồn các trang này ra thư mục `Areas/Identity/Pages/Account` của dự án. Nhờ đó, đồ án có thể can thiệp trực tiếp để tinh chỉnh giao diện trang Đăng nhập trở nên cân đối, loại bỏ các mục đăng nhập bên thứ ba dư thừa, đồng thời khóa tính năng tự đăng ký tự do trên trang `Register` bằng cách điều hướng sang trang thông báo bảo trì hệ thống (*Under Construction*).

2.5. Công nghệ thiết kế giao diện và công cụ quản lý mã nguồn

2.5.1. Ngôn ngữ đánh dấu HTML5, CSS3 và bộ khung giao diện Bootstrap 5

Giao diện phía người dùng (Frontend) của ứng dụng **HotelManager** được xây dựng trên nền tảng các tiêu chuẩn web hiện đại:

- **HTML5 (HyperText Markup Language):** Sử dụng các thẻ ngữ nghĩa (Semantic Tags như `<header>`, `<nav>`, `<main>`, `<footer>`, `<section>`) để định hình cấu trúc bố cục trang web rõ ràng, thân thiện với các trình duyệt hiện đại.
- **CSS3 (Cascading Style Sheets):** Chịu trách nhiệm định dạng màu sắc, phông chữ, khoảng cách lề, hiệu ứng đổ bóng (Box-shadow) và thiết kế tấm ảnh bìa lớn (Hero Banner) trên trang chủ.

- **Bootstrap 5:** Là bộ khung CSS (CSS Framework) phổ biến nhất thế giới hiện nay. Bootstrap 5 cung cấp hệ thống lưới đáp ứng 12 cột (Responsive Grid System với các lớp `container`, `row`, `col-md-*`), cùng hàng loạt các thành phần giao diện dựng sẵn như thanh điều hướng (`navbar`), bảng dữ liệu (`table`), thẻ thông tin (`card`), nút bấm (`btn`) và biểu mẫu nhập liệu (`form-control`). Nhờ có Bootstrap 5, giao diện của ứng dụng có khả năng tự động co giãn và hiển thị đẹp mắt trên nhiều kích thước màn hình khác nhau từ máy tính để bàn đến thiết bị di động.

2.5.2. Chủ đề giao diện Bootswatch Lux trong thiết kế trải nghiệm người dùng

Mặc định, giao diện gốc của Bootstrap 5 mang phong cách trung tính và khá phổ thông, chưa làm nổi bật được nét đặc trưng của ngành dịch vụ khách sạn cao cấp. Để nâng tầm trải nghiệm thị giác (UI/UX), đồ án đã tích hợp chủ đề **Lux** từ thư viện mã nguồn mở **Bootswatch** kết hợp với các tùy biến riêng trong tệp `site.css`:

- **Phong cách thiết kế tối giản và sang trọng:** Chủ đề *Bootswatch Lux* lấy cảm hứng từ các thương hiệu thời trang và khách sạn hạng sang với tông màu chủ đạo đen - trắng tương phản mạnh, các góc cạnh vuông vức, hiện đại và hệ thống phông chữ không chân (Sans-serif) sắc nét, viết hoa tinh tế ở các tiêu đề và nút điều hướng.
- **Tối ưu hóa khả năng đọc và độ tương phản:** Kết hợp giữa chủ đề Lux và các quy tắc CSS tùy chỉnh, hệ thống đảm bảo các văn bản hiển thị trên nền tối (như dòng chữ chào mừng trên tấm ảnh bìa Hero Banner hoặc thanh điều hướng) luôn có độ tương phản cao, rõ ràng và mang lại cảm giác chuyên nghiệp cho khách hàng khi đặt phòng.

2.5.3. Môi trường phát triển và công cụ quản lý phiên bản Git / GitHub Desktop

Để đảm bảo quy trình xây dựng phần mềm diễn ra khoa học, an toàn và tuân thủ chuẩn mực công nghiệp, đồ án sử dụng bộ công cụ phát triển và quản lý mã nguồn sau:

- **Trình biên tập mã nguồn Visual Studio Code (VS Code):** Một môi trường phát triển tích hợp nhẹ, mạnh mẽ của Microsoft hỗ trợ biên dịch, gợi ý mã lệnh thông minh (IntelliSense) và gỡ lỗi (Debugging) cho ngôn ngữ C# và các tệp giao diện Razor[cite: 1].
- **Hệ thống quản lý phiên bản phân tán Git và nền tảng GitHub:** Git giúp theo dõi toàn bộ lịch sử thay đổi của mã nguồn theo từng giai đoạn phát triển (Commits) trên nhánh chính (`main`)[cite: 1]. Thông qua tệp cấu hình `.gitignore` chuẩn của .NET, hệ thống tự động loại bỏ các tệp biên dịch tạm thời (`bin/`, `obj/`), chỉ lưu trữ mã nguồn thuần túy.
- **Ứng dụng GitHub Desktop:** Thay vì thao tác hoàn toàn bằng các câu lệnh phức tạp trên cửa sổ dòng lệnh, việc sử dụng công cụ giao diện đồ họa **GitHub Desktop** giúp quản lý kho lưu trữ cục bộ (Local Repository), kiểm tra trực quan các tệp vừa chỉnh sửa, mở nhanh dự án trong Visual Studio Code hoặc trình quản lý tệp Windows Explorer, và đồng bộ hóa (Push/Fetch) mã nguồn lên đám mây GitHub một cách an toàn, nhanh chóng[cite: 1]. Đồng thời, nền tảng GitHub cho phép thiết lập quyền cộng tác viên (Collaborators) để chia sẻ mã nguồn trực tiếp với giảng viên hướng dẫn phục vụ công tác kiểm tra và đánh giá đồ án.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Phân tích yêu cầu hệ thống

Quá trình hiện thực hóa hệ thống quản lý và đặt phòng khách sạn **HotelManager** bắt đầu bằng việc khảo sát, phân tích và đặc tả chi tiết các yêu cầu phần mềm. Đây là bước nền tảng nhằm xác định rõ hệ thống cần phục vụ những đối tượng nào, cung cấp những tính năng gì và phải đáp ứng các tiêu chuẩn kỹ thuật ra sao trước khi tiến hành thiết kế và lập trình.

3.1.1. Xác định các tác nhân của hệ thống (Actors)

Trong mô hình hóa hệ thống theo ngôn ngữ UML (Unified Modeling Language), tác nhân (Actor) là một thực thể bên ngoài tương tác trực tiếp với hệ thống. Qua phân tích bài toán thực tế, hệ thống **HotelManager** bao gồm 3 tác nhân chính có mối quan hệ phân cấp:

Bảng 3.1. Danh sách các tác nhân trong hệ thống HotelManager

STT	Tên tác nhân	Vai trò trong hệ thống	Mô tả chi tiết quyền hạn và phạm vi hoạt động
1	**Khách vãng lai (Guest)**	Người dùng chưa xác thực	Là người dùng truy cập vào trang web qua trình duyệt nhưng chưa đăng nhập. Được phép xem Trang chủ (Hero Banner), xem danh sách các phòng hiện có của

STT	Tên tác nhân	Vai trò trong hệ thống	Mô tả chi tiết quyền hạn và phạm vi hoạt động
			khách sạn, xem trang Chính sách bảo mật và truy cập trang Đăng nhập. Khi bấm vào nút Đặt phòng hoặc Đăng ký tài khoản mới, hệ thống sẽ chặn hoặc điều hướng sang trang thông báo tương ứng.
2	**Khách hàng (Client)**	Người dùng đã xác thực (`Role: Client`)	Kế thừa toàn bộ quyền của Khách vãng lai. Sau khi đăng nhập bằng tài khoản khách hàng (ví dụ: `client@hotel.com`), người dùng có quyền thực hiện đặt phòng trực tuyến (chọn phòng, chọn ngày nhận và trả phòng) và truy cập trang **My Bookings** để xem danh sách các đơn đặt phòng của riêng mình.

STT	Tên tác nhân	Vai trò trong hệ thống	Mô tả chi tiết quyền hạn và phạm vi hoạt động
3	**Quản trị viên (Administrator)**	Quản lý khách sạn (`Role: Admin`)	Kế thừa quyền xác thực của hệ thống. Sau khi đăng nhập bằng tài khoản quản trị (ví dụ: `boss@hotel.com`), người dùng có toàn quyền quản lý danh mục phòng (xem danh sách, thêm phòng mới vào cơ sở dữ liệu) và truy cập trang quản lý toàn bộ đơn đặt phòng của tất cả khách hàng trong hệ thống.

3.1.2. Yêu cầu chức năng (Functional Requirements)

Các yêu cầu chức năng mô tả hành vi và các dịch vụ cụ thể mà phần mềm phải cung cấp cho người dùng:

1. **Nhóm chức năng Xác thực và Quản lý phiên làm việc:**

    - Cho phép người dùng đăng nhập vào hệ thống bằng địa chỉ Email và Mật khẩu thông qua giao diện đã được tối ưu hóa bố cục.

- Tự động khởi tạo dữ liệu gốc (Data Seeding) bao gồm hai nhóm quyền (`Admin`, `Client`) và hai tài khoản mẫu ngay khi khởi chạy ứng dụng lần đầu.
- Hiển thị trạng thái đăng nhập động trên thanh điều hướng (hiển thị Email người dùng đang đăng nhập và nút Đăng xuất).
- Kiểm soát tính năng tự đăng ký tài khoản mới: Khi người dùng truy cập vào đường dẫn đăng ký (`Register`), hệ thống tự động chuyển hướng sang trang thông báo hệ thống đang xây dựng (*Under Construction*) nhằm ngăn chặn việc tạo tài khoản tự do trái phép.

2. **Nhóm chức năng Quản lý danh mục phòng nghỉ (Room Management):**

- Hiển thị danh sách toàn bộ các phòng nghỉ của khách sạn kèm theo các thông tin: Số phòng/Tên phòng, Loại phòng, Giá thuê một đêm và Trạng thái phòng.
- Cung cấp biểu mẫu (Form) chỉ dành riêng cho Quản trị viên để nhập thông tin và lưu một phòng nghỉ mới vào cơ sở dữ liệu SQL Server.
- Kiểm tra tính hợp lệ của dữ liệu đầu vào (Data Validation) khi thêm phòng mới (không được để trống tên phòng, mức giá phải là số hợp lệ).

3. **Nhóm chức năng Đặt phòng và Quản lý đơn đặt phòng (Booking Management):**

- Cho phép người dùng đã đăng nhập chọn phòng nghỉ từ danh sách, nhập ngày nhận phòng (Check-in Date) và ngày trả phòng (Check-out Date) để tạo đơn đặt phòng mới.

- Tự động ghi nhận thông tin định danh của người đặt (User ID / Email đang đăng nhập) gắn liền với đơn đặt phòng mà không bắt người dùng phải tự nhập thủ công, tránh giả mạo thông tin.
- Cung cấp trang **My Bookings** cho phép Khách hàng tra cứu lịch sử các phòng do chính tài khoản của mình đã đặt.
- Cung cấp trang **All Bookings** cho phép Quản trị viên xem tổng hợp toàn bộ đơn đặt phòng trên hệ thống kèm thông tin phòng và tài khoản khách đặt.

3.1.3. Yêu cầu phi chức năng (Non-Functional Requirements)

Bên cạnh các tính năng nghiệp vụ, hệ thống **HotelManager** phải đáp ứng các tiêu chuẩn chất lượng phần mềm sau:

- **Yêu cầu về bảo mật (Security):**
  - Mật khẩu người dùng tuyệt đối không được lưu dưới dạng văn bản thuần (Plain text) mà phải được mã hóa băm một chiều (Password Hashing) theo chuẩn của ASP.NET Core Identity.
  - Áp dụng cơ chế bảo vệ chống tấn công giả mạo yêu cầu liên trang (Cross-Site Request Forgery - CSRF) trên tất cả các biểu mẫu `HTTP POST` thông qua `ValidateAntiForgeryToken`.
  - Phân quyền hai lớp: Ẩn các liên kết quản trị trên giao diện đối với tài khoản thường, đồng thời chặn cứng tại tầng Controller bằng thuộc tính `[Authorize(Roles = "Admin")]` để chống truy cập trực tiếp qua thanh địa chỉ URL.
- **Yêu cầu về giao diện và trải nghiệm người dùng (UI/UX):**
  - Giao diện đồng bộ, sang trọng với chủ đề **Bootswatch Lux** và phông chữ rõ ràng, độ tương phản cao.

    - Trang chủ có hình ảnh đại diện lớn (Hero Banner) tạo điểm nhấn thị giác ngay khi truy cập.
    - Bố cục đáp ứng (Responsive Design), hiển thị cân đối trên màn hình máy tính xách tay và máy tính để bàn.
- **Yêu cầu về hiệu năng và độ tin cậy (Performance & Reliability):**
    - Thời gian phản hồi các thao tác chuyển trang và truy vấn danh sách phòng dưới 1 giây trong môi trường mạng cục bộ.
    - Đảm bảo tính toàn vẹn tham chiếu (Referential Integrity) giữa bảng Phòng và bảng Đơn đặt phòng trong cơ sở dữ liệu.

3.2. Thiết kế hệ thống

Dựa trên kết quả phân tích yêu cầu ở mục 3.1, đồ án tiến hành thiết kế chi tiết các ca sử dụng (Use Cases), luồng hoạt động nghiệp vụ và trình tự trao đổi thông điệp giữa các thành phần trong kiến trúc MVC.

3.2.1. Phân rã Use Case và đặc tả chi tiết

a) Bảng phân rã chức năng hệ thống theo tác nhân

Cấu trúc tính năng của hệ thống **HotelManager** được phân rã thành 9 ca sử dụng (Use Cases) và ánh xạ trực tiếp theo quyền hạn của từng nhóm tác nhân như sau:

Bảng 3.2. Phân rã các ca sử dụng (Use Cases) theo tác nhân

Tác nhân (Actor)	Mã UC	Tên chức năng (Use Case)	Mô tả phạm vi quyền hạn
Khách vãng lai *(Chưa đăng nhập)*	**UC01**	Xem trang chủ & danh sách phòng	Được phép xem giao diện trang chủ, tham khảo giá và tình trạng phòng trống.
	UC02	Xem thông báo bảo trì khi Đăng ký	Khi bấm vào mục "Register", hệ thống điều hướng sang trang thông báo *Under Construction*.
	UC03	Đăng nhập hệ thống	Sử dụng địa chỉ Email và Mật khẩu để xác thực danh tính vào hệ thống.
Khách hàng *(Role: Client)*	**UC04**	Đặt phòng trực tuyến	Kế thừa quyền khách vãng lai. Chọn phòng nghỉ, chọn ngày nhận/trả và tạo đơn đặt phòng mới.

Tác nhân (Actor)	Mã UC	Tên chức năng (Use Case)	Mô tả phạm vi quyền hạn
	UC05	Xem lịch sử đặt phòng cá nhân	Truy cập trang "My Bookings" để xem danh sách các phòng đã đặt của riêng mình.
	UC06	Đăng xuất tài khoản	Hủy phiên làm việc hiện tại và trở về trạng thái khách vãng lai.
Quản trị viên *(Role: Admin)*	**UC07**	Thêm phòng nghỉ mới	Kế thừa quyền khách vãng lai. Truy cập biểu mẫu thêm phòng mới vào cơ sở dữ liệu.
	UC08	Quản lý danh mục toàn bộ phòng	Xem và kiểm soát toàn bộ danh sách phòng nghỉ hiện có trong khách sạn.
	UC09	Xem toàn bộ đơn đặt phòng	Quản lý, giám sát thông tin đặt phòng

Tác nhân (Actor)	Mã UC	Tên chức năng (Use Case)	Mô tả phạm vi quyền hạn
			của tất cả khách hàng trên hệ thống.
	UC06	Đăng xuất tài khoản	Hủy phiên làm việc quản trị.

b) Các bảng đặc tả Use Case trọng tâm (Use Case Specifications)

Bảng 3.3. Đặc tả Use Case UC03 – Đăng nhập hệ thống

Mục đặc tả	Nội dung chi tiết
Mã và Tên Use Case	**UC03: Đăng nhập hệ thống (Login)**
Tác nhân chính	Khách hàng (`Client`), Quản trị viên (`Admin`).
Mô tả tóm tắt	Cho phép người dùng xác thực danh tính bằng Email và Mật khẩu để kích hoạt phiên làm việc theo đúng vai trò được phân quyền.
Điều kiện tiên quyết (Pre-conditions)	Người dùng chưa đăng nhập; cơ sở dữ liệu đã được khởi tạo sẵn các tài khoản thông qua cơ chế Data Seeding.

Mục đặc tả	Nội dung chi tiết
Điều kiện sau (Post-conditions)	Hệ thống tạo Cookie xác thực trên trình duyệt và chuyển hướng người dùng về Trang chủ với thanh điều hướng tương ứng với quyền hạn.
Luồng sự kiện chính (Basic Flow)	1. Người dùng bấm vào nút **Login** trên thanh điều hướng. 2. Hệ thống hiển thị trang biểu mẫu Đăng nhập đã được căn giữa màn hình. 3. Người dùng nhập địa chỉ **Email** và **Password**, sau đó bấm nút **Log in**. 4. Hệ thống (`SignInManager`) kiểm tra thông tin với bảng `AspNetUsers`. 5. Thông tin hợp lệ, hệ thống ghi nhận phiên đăng nhập và chuyển hướng về trang chủ (`Home/Index`).
Luồng sự kiện rẽ nhánh (Alternative Flow)	**4a. Sai Email hoặc Mật khẩu:** Hệ thống giữ nguyên trang Đăng nhập và hiển thị thông báo lỗi *"Invalid login attempt."* màu đỏ để người dùng nhập lại.

Mục đặc tả	Nội dung chi tiết
	3a. Bấm vào liên kết Register: Hệ thống chuyển hướng người dùng sang trang thông báo *Under Construction*.

Bảng 3.4. Đặc tả Use Case UC04 – Đặt phòng trực tuyến

Mục đặc tả	Nội dung chi tiết
Mã và Tên Use Case	**UC04: Đặt phòng trực tuyến (Create Booking)**
Tác nhân chính	Khách hàng đã đăng nhập (`Client` hoặc `Admin`).
Mô tả tóm tắt	Cho phép khách hàng chọn một phòng nghỉ và đăng ký khoảng thời gian lưu trú (Check-in / Check-out) để lưu đơn đặt phòng vào hệ thống.
Điều kiện tiên quyết	Người dùng phải ở trạng thái đã đăng nhập; trong hệ thống đã có ít nhất một phòng nghỉ đang mở bán.

Mục đặc tả	Nội dung chi tiết
Điều kiện sau	Một bản ghi mới được lưu vào bảng `Bookings` trong cơ sở dữ liệu, gắn liền với `RoomId` và tài khoản của người dùng hiện tại.
Luồng sự kiện chính	1. Từ danh sách phòng hoặc thanh điều hướng, người dùng chọn chức năng **Book a Room**. 2. Hệ thống kiểm tra trạng thái đăng nhập (hợp lệ) và hiển thị biểu mẫu đặt phòng. 3. Người dùng chọn phòng nghỉ từ danh sách thả xuống (Dropdown), chọn **Ngày nhận phòng** và **Ngày trả phòng**, sau đó bấm nút **Confirm / Create**. 4. Hệ thống tự động lấy thông tin định danh của người dùng đang đăng nhập từ phiên làm việc (`User.Identity`). 5. Hệ thống kiểm tra tính hợp lệ của dữ liệu, lưu đơn đặt phòng vào cơ sở dữ liệu thông qua Entity Framework Core. 6. Hệ thống chuyển hướng người dùng sang trang danh sách đơn đặt phòng (**My Bookings**) để xác nhận kết quả.

Mục đặc tả	Nội dung chi tiết
Luồng sự kiện ngoại lệ (Exception Flow)	**2a. Người dùng chưa đăng nhập:** Middleware xác thực tự động chặn yêu cầu và chuyển hướng trình duyệt về trang `/Identity/Account/Login`. **5a. Dữ liệu ngày tháng bỏ trống hoặc không hợp lệ:** Hệ thống báo lỗi kiểm tra dữ liệu ngay dưới ô nhập liệu và yêu cầu người dùng điền đầy đủ.

Bảng 3.5. Đặc tả Use Case UC05 – Xem lịch sử đặt phòng cá nhân (My Bookings)

Mục đặc tả	Nội dung chi tiết
Mã và Tên Use Case	**UC05: Xem lịch sử đặt phòng cá nhân (My Bookings)**
Tác nhân chính	Khách hàng (`Client`).
Mô tả tóm tắt	Hiển thị danh sách các đơn đặt phòng do chính tài khoản khách hàng đang đăng nhập thực hiện, đảm bảo tính bảo mật riêng tư.

Mục đặc tả	Nội dung chi tiết
Điều kiện tiên quyết	Khách hàng đã đăng nhập thành công vào hệ thống.
Luồng sự kiện chính	1. Khách hàng bấm vào mục **My Bookings** trên thanh điều hướng. 2. Hệ thống truy xuất mã định danh/Email của người dùng hiện tại. 3. Hệ thống thực thi câu lệnh truy vấn LINQ lọc trên bảng `Bookings` với điều kiện khớp thông tin người dùng hiện tại, đồng thời kết nối (`Include`) với bảng `Rooms` để lấy tên phòng và giá phòng. 4. Hệ thống trả về giao diện bảng danh sách các đơn đặt phòng của khách hàng đó.
Luồng sự kiện rẽ nhánh	**4a. Khách hàng chưa từng đặt phòng nào:** Giao diện hiển thị bảng trống hoặc thông báo chưa có đơn đặt phòng nào được ghi nhận.

Bảng 3.6. Đặc tả Use Case UC07 – Thêm phòng nghỉ mới (Dành cho Quản trị viên)

Mục đặc tả	Nội dung chi tiết
Mã và Tên Use Case	**UC07: Thêm phòng nghỉ mới vào hệ thống (Create Room)**
Tác nhân chính	Quản trị viên (`Admin`).
Mô tả tóm tắt	Cho phép người quản lý khách sạn bổ sung thêm phòng nghỉ mới kèm mức giá và mô tả vào cơ sở dữ liệu.
Điều kiện tiên quyết	Người dùng phải đăng nhập bằng tài khoản có vai trò `Admin` (`boss@hotel.com`).
Điều kiện sau	Phòng nghỉ mới được lưu vào bảng `Rooms` và hiển thị ngay lập tức trên trang danh sách phòng cho khách hàng xem và đặt.
Luồng sự kiện chính	1. Quản trị viên truy cập mục quản lý phòng (**Rooms**) và bấm nút **Create New** (Thêm phòng mới). 2. Hệ thống kiểm tra nhãn `[Authorize(Roles = "Admin")]` tại Controller.

Mục đặc tả	Nội dung chi tiết
	3. Hệ thống hiển thị biểu mẫu nhập thông tin phòng mới (Số/Tên phòng, Loại phòng, Giá mỗi đêm, Tình trạng). 4. Quản trị viên nhập đầy đủ thông tin và bấm nút **Create**. 5. Hệ thống kiểm tra tính hợp lệ (`ModelState.IsValid`), thêm đối tượng `Room` vào `ApplicationDbContext` và gọi `SaveChangesAsync()`. 6. Hệ thống chuyển hướng về trang danh sách phòng.
Luồng sự kiện ngoại lệ	**2a. Tài khoản Client cố tình truy cập đường dẫn `/Rooms/Create`:** Hệ thống từ chối quyền truy cập và chuyển hướng sang trang báo lỗi **Access Denied** (Từ chối truy cập).

3.2.2. Đặc tả luồng hoạt động hệ thống (Activity Flows)

Luồng điều khiển từng bước giữa Người dùng và Hệ thống trong các nghiệp vụ quan trọng được đặc tả chi tiết thông qua hai bảng luồng hoạt động dưới đây:

a) Luồng hoạt động Xác thực và Kiểm soát Đăng ký

Bảng 3.7. Bảng đặc tả luồng hoạt động Đăng nhập và Chặn đăng ký tự do

Bước	**Hành động của Người dùng**	**Xử lý của Hệ thống (HotelManager)**	**Kết quả / Trạng thái chuyển tiếp**
1	Truy cập khu vực Tài khoản trên thanh điều hướng.	Hiển thị các tùy chọn điều hướng **Login** và **Register**.	Chờ người dùng thao tác.
2a	Bấm vào nút **Register** (Đăng ký tài khoản).	Chặn yêu cầu tạo tài khoản mặc định và chuyển hướng đường dẫn nội bộ.	Hiển thị trang thông báo **Under Construction** (Khóa đăng ký tự do để bảo mật).
2b	Bấm vào nút **Login** (Đăng nhập).	Tải giao diện trang Đăng nhập đã được tinh gọn và căn giữa màn hình.	Hiển thị biểu mẫu nhập Email và Password.

Bước	Hành động của Người dùng	Xử lý của Hệ thống (HotelManager)	Kết quả / Trạng thái chuyển tiếp
3	Nhập Email, Password và bấm nút xác nhận **Log in**.	Gửi yêu cầu xác thực xuống cơ sở dữ liệu (`AspNetUsers`) thông qua `SignInManager`.	Hệ thống đối chiếu thông tin tài khoản và mật khẩu băm.
4a	*(Trường hợp nhập sai thông tin)*	Từ chối xác thực phiên làm việc.	Hiển thị thông báo lỗi màu đỏ *"Invalid login attempt."* và quay lại Bước 3.
4b	*(Trường hợp nhập đúng thông tin)*	Khởi tạo Cookie xác thực và gắn vai trò tương ứng (`Admin` hoặc `Client`).	Chuyển hướng người dùng về Trang chủ (`Home/Index`) thành công.

b) Luồng hoạt động Đặt phòng trực tuyến của Khách hàng

Bảng 3.8. Bảng đặc tả luồng hoạt động Đặt phòng trực tuyến

Bước	Hành động của Khách hàng	Xử lý của Hệ thống (HotelManager)	Kết quả / Trạng thái chuyển tiếp
1	Bấm chọn chức năng **Book a Room** trên giao diện.	Kiểm tra trạng thái xác thực thông qua Middleware `[Authorize]`.	Nếu chưa đăng nhập: Tự động chuyển hướng về trang **Login**.
2	*(Đã đăng nhập hợp lệ)*	Truy vấn danh sách toàn bộ phòng nghỉ từ bảng `Rooms` trong SQL Server.	Hiển thị biểu mẫu Đặt phòng kèm danh sách chọn phòng (Dropdown).
3	Chọn Phòng nghỉ, nhập Ngày nhận phòng, Ngày trả phòng và bấm nút xác nhận.	Tiếp nhận yêu cầu `HTTP POST`. Tự động lấy địa chỉ Email của người dùng đang đăng nhập từ `User.Identity.Name`.	Gắn thông tin định danh khách hàng vào đối tượng `Booking`.

Bước	Hành động của Khách hàng	Xử lý của Hệ thống (HotelManager)	Kết quả / Trạng thái chuyển tiếp
4	*(Chờ hệ thống kiểm tra)*	Kiểm tra tính hợp lệ của dữ liệu đầu vào (`ModelState.IsValid`).	Nếu thiếu dữ liệu: Báo lỗi trên biểu mẫu để khách hàng nhập lại.
5	*(Dữ liệu hợp lệ)*	Thực thi phương thức `_context.Add(booking)` và `SaveChangesAsync()`.	Bản ghi đơn đặt phòng mới được lưu bền vững vào bảng `Bookings`.
6	Xem danh sách đơn đặt phòng vừa tạo.	Thực thi lệnh chuyển hướng `RedirectToAction("MyBookings")`.	Hiển thị trang **My Bookings** với đơn đặt phòng mới nhất của khách hàng.

3.2.3. Thiết kế trình tự trao đổi thông điệp (Sequence Flow)

Quá trình trao đổi thông điệp theo trình tự thời gian giữa các thành phần trong kiến trúc **ASP.NET Core MVC** (`Giao diện View`, `BookingsController`, `ApplicationDbContext` và `SQL Server DB`) khi thực hiện chức năng Đặt phòng được đặc tả trong bảng sau:

Bảng 3.9. Bảng đặc tả trình tự trao đổi thông điệp chức năng Đặt phòng trong kiến trúc MVC

STT	Thành phần gửi	Thành phần nhận	Tên thông điệp / Lời gọi phương thức	Mục đích và nội dung xử lý
1	Khách hàng	Giao diện View	Yêu cầu `HTTP GET /Bookings/Create`	Người dùng nhấn vào nút Đặt phòng trên giao diện.
2	Giao diện View	`BookingsController`	Gọi phương thức `Create() [GET]`	Bộ điều khiển tiếp nhận yêu cầu khởi tạo biểu mẫu đặt phòng.
3	`BookingsController`	`ApplicationDbContext`	Truy vấn `_context.Rooms`	Yêu cầu lấy toàn bộ danh mục phòng

STT	Thành phần gửi	Thành phần nhận	Tên thông điệp / Lời gọi phương thức	Mục đích và nội dung xử lý
				hiện có thông qua EF Core.
4	`Application DbContext`	SQL Server DB	Thực thi `SELECT * FROM Rooms`	EF Core dịch truy vấn LINQ sang câu lệnh SQL và gửi tới CSDL.
5	SQL Server DB	Giao diện View	Trả danh sách `SelectList` về View	Đổ danh sách tên/số phòng vào thẻ `<select>` trên biểu mẫu.
6	Khách hàng	Giao diện View	Nhập thông tin & bấm Submit	Khách hàng chọn phòng, chọn ngày Check-in/Check-out và gửi biểu mẫu.

STT	Thành phần gửi	Thành phần nhận	Tên thông điệp / Lời gọi phương thức	Mục đích và nội dung xử lý
7	Giao diện View	`BookingsController`	Gửi `HTTP POST Create(Booking)`	Truyền dữ liệu biểu mẫu kèm mã chống giả mạo (`AntiForgeryToken`) lên máy chủ.
8	`BookingsController`	`BookingsController`	Gán định danh & kiểm tra `ModelState`	Tự động gán `booking.UserEmail = User.Identity.Name` và kiểm tra tính hợp lệ.
9	`BookingsController`	`ApplicationDbContext`	Gọi `_context.Add()` & `SaveChangesAsync()`	Đưa đối tượng `Booking` vào trạng thái chờ ghi và yêu cầu lưu xuống CSDL.

STT	Thành phần gửi	Thành phần nhận	Tên thông điệp / Lời gọi phương thức	Mục đích và nội dung xử lý
10	`ApplicationDbContext`	SQL Server DB	Thực thi `INSERT INTO Bookings`	Ghi bền vững bản ghi đơn đặt phòng mới vào bảng vật lý trong SQL Server.
11	`BookingsController`	Khách hàng	`RedirectToAction("MyBookings")`	Điều hướng trình duyệt của khách hàng sang trang Lịch sử đặt phòng cá nhân.

3.3. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống **HotelManager** được xây dựng trên hệ quản trị **Microsoft SQL Server (LocalDB)** thông qua phương pháp **Code-First** của Entity Framework Core. Toàn bộ cấu trúc cơ sở dữ liệu là sự kết hợp chặt chẽ giữa các bảng nghiệp vụ khách sạn (`Rooms`, `Bookings`) và hệ thống các bảng bảo mật tiêu chuẩn của **ASP.NET Core Identity**.

3.3.1. Đặc tả quan hệ thực thể (Entity Relationship - ERD)

Cấu trúc liên kết và ràng buộc tham chiếu giữa các bảng thực thể cốt lõi trong cơ sở dữ liệu được mô tả chi tiết qua bảng định nghĩa quan hệ dưới đây:

Bảng 3.10. Bảng đặc tả các mối quan hệ thực thể trong cơ sở dữ liệu HotelManager

Bảng dữ liệu gốc (Thực thể 1)	Loại quan hệ	Bảng tham chiếu (Thực thể N)	Khóa liên kết (Primary Key ➔ Foreign Key)	Ý nghĩa nghiệp vụ và ràng buộc toàn vẹn
AspNetRoles *(Bảng Vai trò)*	**1 - N** *(Một - Nhiều)*	**AspNetUserRoles** *(Bảng Phân quyền)*	Id (AspNetRoles) ➔ RoleId (AspNetUserRoles)	Một vai trò trong hệ thống (Admin hoặc Client) có thể được cấp cho nhiều tài khoản người dùng khác nhau.
AspNetUsers *(Bảng Người dùng)*	**1 - N** *(Một - Nhiều)*	**AspNetUserRoles** *(Bảng Phân quyền)*	Id (AspNetUsers) ➔ UserId (AspNetUserRoles)	Một tài khoản người dùng có thể nắm giữ một hoặc nhiều vai trò thông qua bảng trung gian AspNetUserRoles.

Bảng dữ liệu gốc (Thực thể 1)	Loại quan hệ	Bảng tham chiếu (Thực thể N)	Khóa liên kết (Primary Key ➔ Foreign Key)	Ý nghĩa nghiệp vụ và ràng buộc toàn vẹn
Rooms *(Bảng Phòng nghỉ)*	**1 - N** *(Một - Nhiều)*	**Bookings** *(Bảng Đơn đặt phòng)*	Id (Rooms) ➔ RoomId (Bookings)	Một phòng nghỉ của khách sạn có thể xuất hiện trong nhiều đơn đặt phòng khác nhau theo thời gian. Không thể đặt một phòng không tồn tại trong bảng Rooms.
AspNetUsers *(Bảng Người dùng)*	**1 - N** *(Một - Nhiều)*	**Bookings** *(Bảng Đơn đặt phòng)*	Email / Id (AspNetUsers) ➔ UserEmail (Bookings)	Một khách hàng sau khi đăng nhập có thể thực hiện tạo nhiều đơn đặt phòng khác nhau và tra cứu lại theo định danh của mình.

3.3.2. Đặc tả chi tiết các bảng dữ liệu (Data Dictionary)

Dưới đây là đặc tả chi tiết cấu trúc các bảng vật lý được khởi tạo trong SQL Server thông qua công cụ Entity Framework Core Migrations:

a) Bảng Rooms (Lưu trữ thông tin danh mục phòng nghỉ)

Bảng 3.11. Cấu trúc dữ liệu bảng Rooms

STT	Tên trường (Column)	Kiểu dữ liệu (SQL)	Ràng buộc	Mô tả ý nghĩa
1	**Id**	int	PRIMARY KEY, IDENTITY(1,1)	Mã định danh duy nhất của mỗi phòng nghỉ (tự động tăng).
2	**RoomNumber** / **Name**	nvarchar(100)	NOT NULL	Số phòng hoặc tên gọi của phòng nghỉ (ví dụ: *Room 101* - *Deluxe*, *Suite 202*).
3	**RoomType** / **Description**	nvarchar(250)	NULL / NOT NULL	Phân loại hạng phòng (Standard, Deluxe, VIP Suite) và

STT	Tên trường (Column)	Kiểu dữ liệu (SQL)	Ràng buộc	Mô tả ý nghĩa
				mô tả tiện nghi.
4	**Price**	decimal(18, 2)	NOT NULL	Giá thuê phòng tính trên một đêm lưu trú.
5	**IsAvailable**	bit	NOT NULL (Mặc định: 1)	Trạng thái phòng hiện tại (1 - True: Sẵn sàng đón khách; 0 - False: Đang bận/Bảo trì).

b) Bảng Bookings (Lưu trữ thông tin đơn đặt phòng của khách hàng)

Bảng 3.12. Cấu trúc dữ liệu bảng Bookings

STT	Tên trường (Column)	Kiểu dữ liệu (SQL)	Ràng buộc	Mô tả ý nghĩa
1	**Id**	int	PRIMARY KEY, IDENTITY(1,1)	Mã định danh duy nhất của mỗi đơn đặt phòng (tự động tăng).
2	**RoomId**	int	FOREIGN KEY, NOT NULL	Khóa ngoại tham chiếu trực tiếp đến cột Id của bảng Rooms.
3	**UserId** / **UserEmail**	nvarchar(450)	NOT NULL	Lưu thông tin định danh (ID hoặc địa chỉ Email) của tài khoản khách hàng đã thực

STT	Tên trường (Column)	Kiểu dữ liệu (SQL)	Ràng buộc	Mô tả ý nghĩa
				hiện đặt phòng.
4	CheckInDate	datetime2	NOT NULL	Ngày và giờ khách hàng dự kiến nhận phòng (Check-in).
5	CheckOutDate	datetime2	NOT NULL	Ngày và giờ khách hàng dự kiến trả phòng (Check-out).

c) Bảng AspNetUsers (Lưu trữ tài khoản người dùng của Identity)

Bảng 3.13. Cấu trúc dữ liệu bảng AspNetUsers

STT	Tên trường (Column)	Kiểu dữ liệu (SQL)	Ràng buộc	Mô tả ý nghĩa
1	**Id**	nvarchar(450)	PRIMARY KEY	Mã định danh người dùng dạng chuỗi GUID sinh tự động.
2	**UserName**	nvarchar(256)	NULL	Tên đăng nhập của người dùng (mặc định trùng với địa chỉ Email).
3	**Email**	nvarchar(256)	NULL	Địa chỉ Email của tài khoản (ví dụ: boss@hotel.com, client@hotel.com).
4	**EmailConfirmed**	bit	NOT NULL	Trạng thái xác nhận Email (1: Đã xác nhận).
5	**PasswordHash**	nvarchar(max)	NULL	Chuỗi mật khẩu đã được mã hóa băm an toàn tuyệt đối.
6	**SecurityStamp**	nvarchar(max)	NULL	Giá trị ngẫu nhiên thay đổi mỗi khi thông tin bảo mật của tài khoản cập nhật.

d) Bảng `AspNetRoles` và `AspNetUserRoles` (Lưu trữ vai trò và phân quyền)

Bảng 3.14. Cấu trúc dữ liệu bảng AspNetRoles và AspNetUserRoles

Tên bảng	Tên trường chính	Kiểu dữ liệu	Ràng buộc	Mô tả ý nghĩa
AspNetRoles	Id	nvarchar(450)	PRIMARY KEY	Mã định danh của vai trò (GUID).
AspNetRoles	Name	nvarchar(256)	NOT NULL	Tên vai trò trong hệ thống: **Admin** (Quản lý) và **Client** (Khách hàng).
AspNetUserRoles	UserId	nvarchar(450)	PK, FOREIGN KEY	Tham chiếu đến cột Id của bảng AspNetUsers.
AspNetUserRoles	RoleId	nvarchar(450)	PK, FOREIGN KEY	Tham chiếu đến cột Id của bảng AspNetRoles.

3.4. Hiện thực hóa và cài đặt chương trình

Phần này trình bày chi tiết cách tổ chức mã nguồn và giải thích các đoạn mã C# cốt lõi đã được cài đặt để biến các bản thiết kế ở mục 3.2 và 3.3 thành sản phẩm phần mềm hoàn chỉnh.

3.4.1. Tổ chức cấu trúc thư mục dự án ASP.NET Core MVC

Mã nguồn của dự án **HotelManager** được tổ chức tuân thủ chặt chẽ cấu trúc tiêu chuẩn của một ứng dụng ASP.NET Core MVC tích hợp Identity Scaffolding. Chi tiết vai trò của từng thư mục và tệp tin chính được trình bày trong bảng dưới đây:

Bảng 3.15. Bảng mô tả cấu trúc thư mục và tệp tin của dự án HotelManager

Thư mục / Tệp tin gốc	Các tệp tin thành phần chính	Chức năng và vai trò trong dự án
`Areas/Identity/`	`Login.cshtml`, `Register.cshtml`, `UnderConstruction.cshtml`	Chứa các trang giao diện xác thực đã được trích xuất (Scaffolded): trang Đăng nhập căn giữa, xử lý khóa tính năng tự đăng ký và hiển thị trang thông báo bảo trì.
`Controllers/`	`HomeController.cs`, `RoomsController.cs`, `BookingsController.cs`	Tầng điều khiển trung tâm: tiếp nhận yêu cầu HTTP, kiểm tra phân quyền `[Authorize]`,

Thư mục / Tệp tin gốc	Các tệp tin thành phần chính	Chức năng và vai trò trong dự án
		truy vấn dữ liệu và điều phối trả về các View tương ứng.
Data/	ApplicationDbContext.cs, thư mục Migrations/	Chứa lớp ngữ cảnh kết nối C# với SQL Server và các tệp lịch sử khởi tạo cấu trúc bảng cơ sở dữ liệu theo hướng Code-First.
Models/	Room.cs, Booking.cs, ErrorViewModel.cs	Tầng dữ liệu: định nghĩa các lớp thực thể biểu diễn Phòng nghỉ, Đơn đặt phòng và các ràng buộc kiểm tra dữ liệu (Data Annotations).
Views/	Thư mục Home/, Rooms/, Bookings/, Shared/ (_Layout.cshtml, _LoginPartial.cshtml)	Tầng giao diện hiển thị sử dụng Razor Engine: định nghĩa khung bố cục chung tích hợp Bootswatch Lux, trang chủ Hero Banner và các trang danh sách/biểu mẫu.

Thư mục / Tệp tin gốc	Các tệp tin thành phần chính	Chức năng và vai trò trong dự án
wwwroot/	css/site.css, thư mục images/	Lưu trữ các tệp tài nguyên tĩnh của trang web bao gồm định dạng CSS tùy chỉnh và hình ảnh nền khách sạn.
appsettings.json	Tệp cấu hình JSON	Lưu trữ chuỗi kết nối cơ sở dữ liệu (DefaultConnection) trỏ tới Microsoft SQL Server Express LocalDB.
Program.cs	Tệp khởi chạy trung tâm	Cấu hình Dependency Injection, chuỗi Middleware xác thực/phân quyền và thực thi đoạn mã khởi tạo tài khoản tự động (Data Seeding).

3.4.2. Cài đặt tầng Model và Ngữ cảnh dữ liệu (ApplicationDbContext)

a) Xây dựng các lớp thực thể (Room.cs và Booking.cs)

Tại thư mục Models, hai lớp thực thể chính được định nghĩa bằng ngôn ngữ C# kèm theo các thuộc tính ràng buộc dữ liệu (Data Annotations):

```
using System.ComponentModel.DataAnnotations;

using System.ComponentModel.DataAnnotations.Schema;

namespace HotelManager.Models

{

    public class Room

    {

        [Key]

        public int Id { get; set; }

        [Required(ErrorMessage = "Vui lòng nhập số phòng hoặc tên phòng")]

        [Display(Name = "Room Name / Number")]

        public string RoomNumber { get; set; } = string.Empty;

        [Required]
```

```
        [Display(Name = "Room Type")]

        public string RoomType { get; set; } = string.Empty;

        [Required]

        [Column(TypeName = "decimal(18,2)")]

        [Display(Name = "Price per Night ($)")]

        public decimal Price { get; set; }

        [Display(Name = "Available")]

        public bool IsAvailable { get; set; } = true;

    }

}
```

Lớp `Booking.cs` thiết lập khóa ngoại `RoomId` và thuộc tính điều hướng (`Navigation Property`) `Room` để Entity Framework Core tự động tạo mối quan hệ Một - Nhiều (1-N):

```
using System.ComponentModel.DataAnnotations;

namespace HotelManager.Models

{

    public class Booking

    {

        [Key]

        public int Id { get; set; }

        [Required]

        [Display(Name = "Room")]

        public int RoomId { get; set; }

        // Thuộc tính điều hướng liên kết tới bảng Rooms

        public Room? Room { get; set; }
```

```
        [Display(Name = "Customer Email")]

        public string UserEmail { get; set; } = string.Empty;

        [Required]

        [DataType(DataType.Date)]

        [Display(Name = "Check-In Date")]

        public DateTime CheckInDate { get; set; }

        [Required]

        [DataType(DataType.Date)]

        [Display(Name = "Check-Out Date")]

        public DateTime CheckOutDate { get; set; }

    }

}
```

b) Cấu hình lớp `ApplicationDbContext.cs`

Tại thư mục `Data`, lớp `ApplicationDbContext` kế thừa từ `IdentityDbContext` và khai báo hai thuộc tính `DbSet` để ánh xạ hai lớp C# ở trên thành hai bảng vật lý trong SQL Server:

```
using HotelManager.Models;

using Microsoft.AspNetCore.Identity.EntityFrameworkCore;

using Microsoft.EntityFrameworkCore;

namespace HotelManager.Data

{

   public class ApplicationDbContext : IdentityDbContext

   {

      public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)

         : base(options)

      {
```

```
    }

    public DbSet<Room> Rooms { get; set; }

    public DbSet<Booking> Bookings { get; set; }

  }

}
```

3.4.3. Hiện thực hóa cơ chế khởi tạo dữ liệu tự động (Data Seeding) trong `Program.cs`

Một trong những điểm nhấn kỹ thuật quan trọng nhất của đồ án là việc cấu hình dịch vụ Identity hỗ trợ phân quyền (`AddRoles<IdentityRole>()`) và tự động tạo sẵn hai tài khoản mẫu ngay khi ứng dụng khởi chạy lần đầu trong tệp `Program.cs`.

Để thuận tiện cho việc kiểm thử và vận hành demo với mật khẩu ngắn gọn (`admin`), cấu hình `IdentityOptions` được điều chỉnh linh hoạt, sau đó một phạm vi dịch vụ (`IServiceScope`) sẽ kiểm tra và khởi tạo tự động:

```
// Đăng ký dịch vụ Identity kèm theo Role và cấu hình quy tắc mật khẩu

builder.Services.AddDefaultIdentity<IdentityUser>(options => {
```

```
    options.SignIn.RequireConfirmedAccount = false;

    options.Password.RequireDigit = false;

    options.Password.RequireLowercase = false;

    options.Password.RequireNonAlphanumeric = false;

    options.Password.RequireUppercase = false;

    options.Password.RequiredLength = 4;

})

.AddRoles<IdentityRole>()

.AddEntityFrameworkStores<ApplicationDbContext>();
```

Đoạn mã thực thi **Data Seeding** tự động tạo quyền `Admin`, `Client` và hai tài khoản `boss@hotel.com` / `client@hotel.com` khi ứng dụng khởi động:

```
using (var scope = app.Services.CreateScope())
```

```
{
    var roleManager = scope.ServiceProvider
        .GetRequiredService<RoleManager<IdentityRole>>();
    var userManager = scope.ServiceProvider
        .GetRequiredService<UserManager<IdentityUser>>();
    // 1. Khởi tạo các vai trò (Roles) nếu chưa tồn tại
    string[] roleNames = { "Admin", "Client" };
    foreach (var roleName in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(roleName))
        {
            await roleManager.CreateAsync(new IdentityRole(roleName));
```

```
    }

}

// 2. Khởi tạo tài khoản Quản lý (boss@hotel.com / mật khẩu: admin)

var adminEmail = "boss@hotel.com";

var adminUser = await userManager.FindByEmailAsync(adminEmail);

if (adminUser == null)

{

    adminUser = new IdentityUser {

        UserName = adminEmail,

        Email = adminEmail,

        EmailConfirmed = true

    };
```

```
    await userManager.CreateAsync(adminUser, "admin");

    await userManager.AddToRoleAsync(adminUser, "Admin");

}

// 3. Khởi tạo tài khoản Khách hàng (client@hotel.com / mật khẩu: admin)

var clientEmail = "client@hotel.com";

var clientUser = await userManager.FindByEmailAsync(clientEmail);

if (clientUser == null)

{

    clientUser = new IdentityUser {

        UserName = clientEmail,

        Email = clientEmail,

        EmailConfirmed = true
```

```
        };

        await userManager.CreateAsync(clientUser, "admin");

        await userManager.AddToRoleAsync(clientUser, "Client");

    }

}
```

Nhờ đoạn mã này, khi giảng viên hoặc bất kỳ nhà phát triển nào tải dự án từ GitHub về máy tính mới[cite: 1], hệ thống sẽ tự động thiết lập đầy đủ tài khoản và quyền hạn mà không cần phải chạy thêm bất kỳ tập lệnh SQL thủ công nào.

3.4.4. Hiện thực hóa tầng Controller và bảo mật phân quyền (`[Authorize]`)

a) Bảo mật chức năng thêm phòng mới trong `RoomsController.cs`

Trong `RoomsController`, toàn bộ khách truy cập đều có thể gọi phương thức `Index()` để xem danh sách phòng. Tuy nhiên, phương thức `Create()` (cả `GET` và `POST`) được gắn thuộc tính `[Authorize(Roles = "Admin")]` nhằm khóa chặt quyền thêm phòng chỉ dành riêng cho Quản trị viên:

```
// Mọi người dùng đều có thể xem danh sách phòng

public async Task<IActionResult> Index()
```

```
{
    return View(await _context.Rooms.ToListAsync());
}
// CHỈ tài khoản có Role "Admin" mới được truy cập trang thêm phòng mới
[Authorize(Roles = "Admin")]
public IActionResult Create()
{
    return View();
}
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
```

```
public async Task<IActionResult> Create(

    [Bind("Id,RoomNumber,RoomType,Price,IsAvailable")] Room room)

{

    if (ModelState.IsValid)

    {

        _context.Add(room);

        await _context.SaveChangesAsync();

        return RedirectToAction(nameof(Index));

    }

    return View(room);

}
```

b) Xử lý Đặt phòng và tách biệt dữ liệu cá nhân trong `BookingsController.cs`

Tại `BookingsController`, thuộc tính `[Authorize]` được đặt ở cấp độ lớp để bắt buộc người dùng phải đăng nhập trước khi thao tác với đơn đặt phòng. Đặc biệt, hệ thống tách bạch rõ hai phương thức:

- Phương thức `Index()` gắn nhãn `[Authorize(Roles = "Admin")]`: Lấy toàn bộ đơn đặt phòng của mọi khách hàng cho Quản lý xem.
- Phương thức `MyBookings()`: Tự động lấy địa chỉ Email/UserName của người dùng đang đăng nhập thông qua `User.Identity?.Name` và dùng LINQ `.Where()` để chỉ lọc ra các đơn đặt phòng của chính người đó:

```
[Authorize]

public class BookingsController : Controller

{

   private readonly ApplicationDbContext _context;

   public BookingsController(ApplicationDbContext context)

   {

      _context = context;
```

```
}

// Dành riêng cho Admin: Xem toàn bộ đơn đặt phòng của hệ thống

[Authorize(Roles = "Admin")]

public async Task<IActionResult> Index()

{

    var allBookings = _context.Bookings.Include(b => b.Room);

    return View(await allBookings.ToListAsync());

}

// Dành cho Khách hàng: Chỉ xem các phòng do chính mình đã đặt

public async Task<IActionResult> MyBookings()

{

    var currentUserEmail = User.Identity?.Name;
```

```
    var userBookings = _context.Bookings
        .Include(b => b.Room)
        .Where(b => b.UserEmail == currentUserEmail);
    return View(await userBookings.ToListAsync());
}
// Xử lý lưu đơn đặt phòng mới và tự động gán Email người đang đăng nhập
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(
    [Bind("Id,RoomId,CheckInDate,CheckOutDate")] Booking booking)
{
    booking.UserEmail = User.Identity?.Name ?? "Unknown";
```

```
        ModelState.Remove("UserEmail");

        if (ModelState.IsValid)

        {

            _context.Add(booking);

            await _context.SaveChangesAsync();

            return RedirectToAction(nameof(MyBookings));

        }

        ViewData["RoomId"] = new SelectList(

            _context.Rooms, "Id", "RoomNumber", booking.RoomId);

        return View(booking);

    }

}
```

3.4.5. Hiện thực hóa tầng View và tinh chỉnh trải nghiệm người dùng (UI/UX)

Để tạo nên diện mạo sang trọng, hiện đại và khắc phục sự đơn điệu của mẫu giao diện mặc định, tầng View đã được thực hiện các cải tiến kỹ thuật sau:

1. **Tích hợp chủ đề Bootswatch Lux và thanh điều hướng phân quyền (`_Layout.cshtml`):**

    - Thay thế tệp `bootstrap.min.css` mặc định bằng bộ định dạng giao diện **Bootswatch Lux**, mang lại phông chữ sắc nét, tông màu đen - trắng cao cấp.
    - Sử dụng cấu trúc điều kiện `@if (User.IsInRole("Admin"))` và `@if (User.Identity.IsAuthenticated)` ngay trên thanh điều hướng để hiển thị đúng các nút chức năng theo từng đối tượng người dùng.

2. **Thiết kế tấm ảnh bìa lớn (Hero Banner) tại Trang chủ (`Views/Home/Index.cshtml`):**

    - Xây dựng khu vực chào mừng nổi bật với hình nền khách sạn chất lượng cao kết hợp lớp phủ tối màu (Dark Overlay) để làm nổi bật dòng tiêu đề màu trắng sáng và nút kêu gọi hành động (*Explore Our Rooms*), khắc phục triệt để hiện tượng chữ tối màu bị chìm vào nền ảnh.

3. **Tối ưu hóa trang Đăng nhập (`Areas/Identity/Pages/Account/Login.cshtml`):**

- Sau khi thực hiện Identity Scaffolding, mã nguồn trang `Login.cshtml` đã được loại bỏ hoàn toàn cột thông báo *"Use another service to log in"* không cần thiết ở phía bên phải.
- Sử dụng hệ thống lưới Bootstrap (`row justify-content-center` và `col-md-5`) kết hợp thẻ khung (`card shadow`) để đưa biểu mẫu đăng nhập ra chính giữa màn hình một cách cân đối và chuyên nghiệp.

4. **Xây dựng cơ chế chuyển hướng bảo trì cho trang Đăng ký (`Register.cshtml`):**

- Khi người dùng nhấn vào nút **Register**, thay vì cho phép tạo tài khoản tự do hoặc hiển thị trang lỗi hệ thống khô khan, phương thức `OnGet()` của trang đăng ký sẽ điều hướng người dùng tới một giao diện thông báo **Under Construction** (Đang trong quá trình xây dựng) được thiết kế lịch sự kèm nút quay lại trang chủ hoặc trang đăng nhập.

3.4.6. Quản lý phiên bản và đồng bộ mã nguồn với GitHub Desktop

Sau khi hoàn thiện toàn bộ mã nguồn và kiểm tra hoạt động ổn định trên máy cục bộ, dự án được đưa vào hệ thống quản lý phiên bản Git và lưu trữ trực tuyến trên nền tảng GitHub:

- Dự án được khởi tạo kho lưu trữ (Repository) mang mã định danh môn học của sinh viên (`ASPNET-DT24TT80162-PHAMTRUN...`) dưới tài khoản `TimothyButTimIsOkToo` và quản lý trên nhánh chính (`main`) thông qua ứng dụng giao diện đồ họa **GitHub Desktop**[cite: 1].

- Sử dụng tệp cấu hình `.gitignore` chuẩn của Visual Studio / .NET để loại bỏ các thư mục biên dịch tạm thời (`bin/`, `obj/`, `.vs/`), giúp kho lưu trữ gọn nhẹ và sạch sẽ.
- Thông qua bảng điều khiển của **GitHub Desktop**, lập trình viên có thể kiểm soát trạng thái các tệp thay đổi (*No local changes*), mở nhanh mã nguồn trong trình soạn thảo **Visual Studio Code** (*Open in Visual Studio Code*), kiểm tra thư mục gốc trong **Windows Explorer** (*Show in Explorer*) hoặc truy cập trực tiếp kho mã nguồn trên trình duyệt (*View on GitHub*) để phân quyền cộng tác viên (*Collaborators*) cho giảng viên hướng dẫn chấm bài[cite: 1].

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

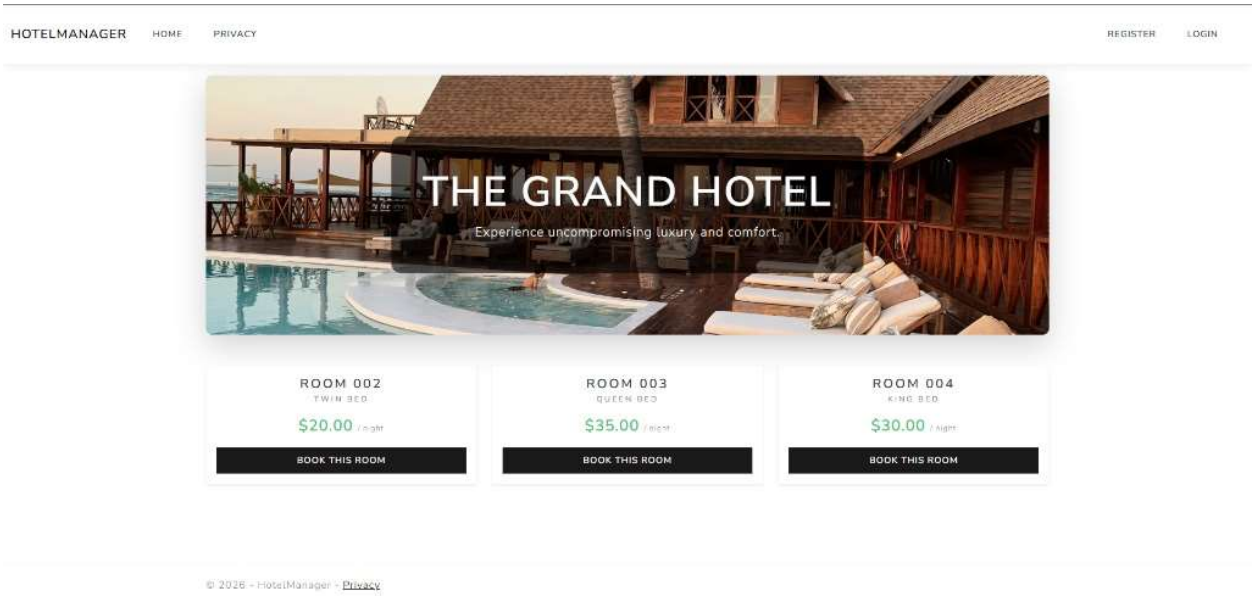
Sau quá trình nghiên cứu lý thuyết, phân tích yêu cầu và hiện thực hóa phần mềm, chương này sẽ trình bày chi tiết các kết quả đạt được của hệ thống **HotelManager**. Nội dung bao gồm việc đánh giá giao diện người dùng thực tế và các kịch bản kiểm thử (Test Cases) nhằm đảm bảo hệ thống hoạt động chính xác, đáp ứng đầy đủ các yêu cầu đã đặt ra ở Chương 3.

4.1. Đánh giá kết quả giao diện người dùng (UI/UX)

Hệ thống đã triển khai thành công giao diện ứng dụng web dựa trên bộ khung Bootstrap 5 kết hợp chủ đề **Bootswatch Lux**. Dưới đây là các kết quả giao diện chức năng nổi bật đã hoàn thiện:

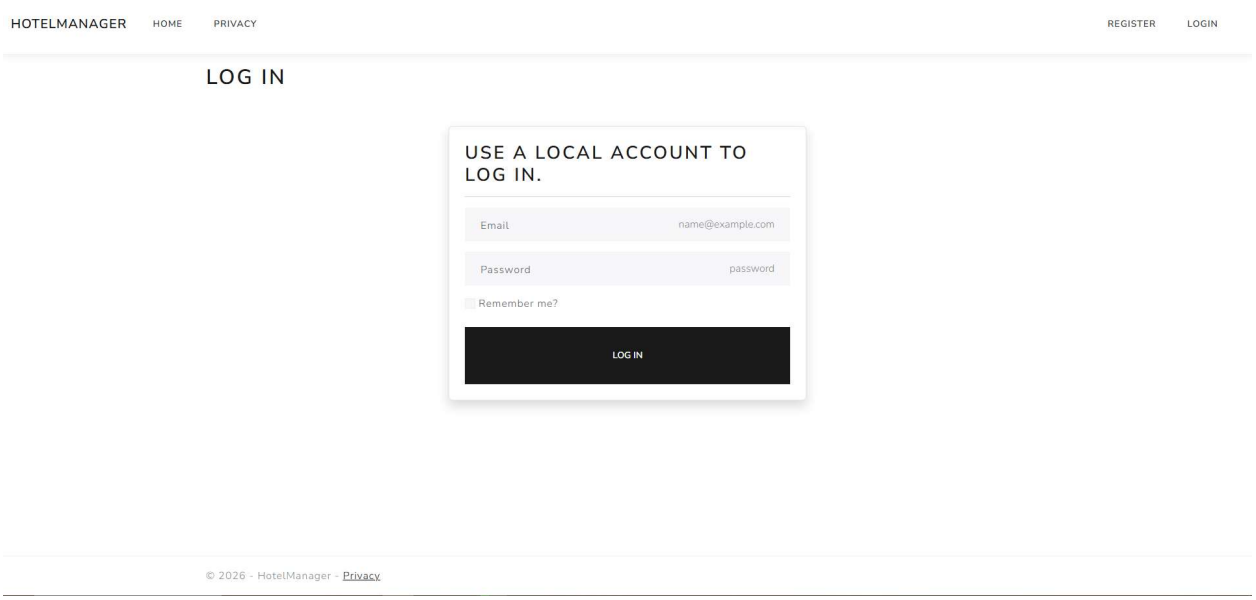
4.1.1. Trang chủ và Danh mục phòng (Khách vãng lai và Khách hàng)

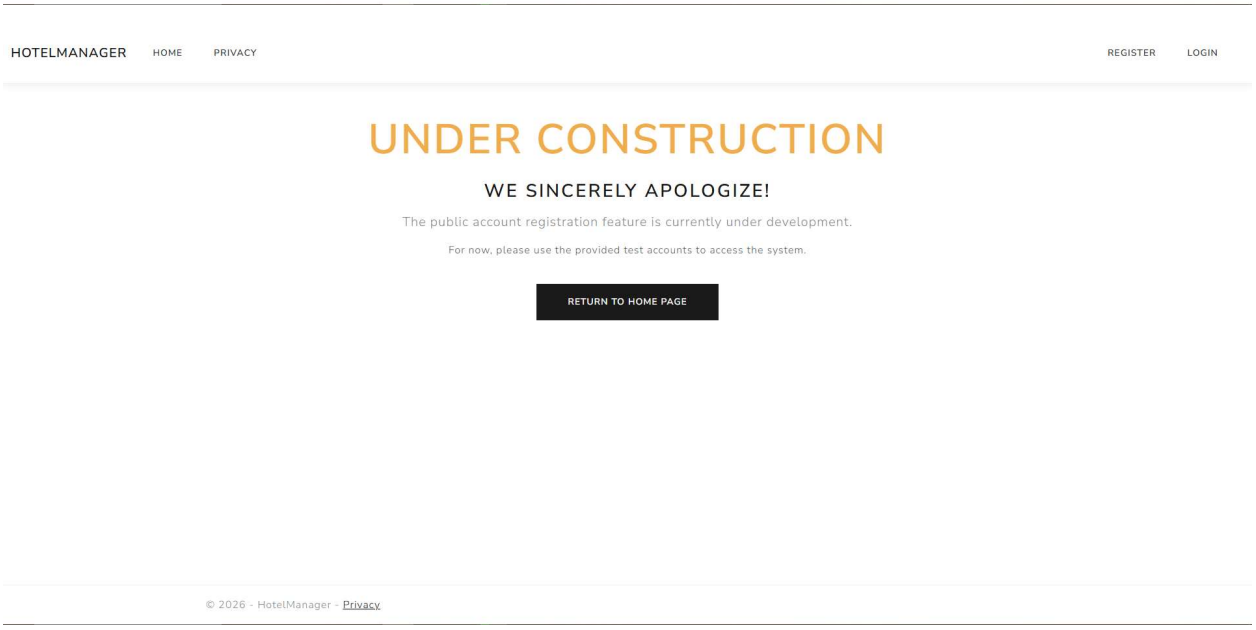
- **Trang chủ (Hero Banner):** Khi người dùng vừa truy cập vào hệ thống, đập vào mắt là một tấm ảnh bìa lớn (Hero Banner) tràn viền chất lượng cao. Việc kết hợp lớp phủ tối màu (Dark Overlay) đã giúp dòng chữ chào mừng và nút kêu gọi hành động (*Explore Our Rooms*) hiển thị với độ tương phản cao nhất. Thanh điều hướng ở trạng thái chưa đăng nhập chỉ hiển thị các nút chức năng công khai (Home, Privacy, Login, Register).
- **Trang danh sách phòng (Rooms Index):** Dữ liệu phòng nghỉ được truy xuất trực tiếp từ cơ sở dữ liệu SQL Server và trình bày dưới dạng bảng tinh gọn. Khách hàng có thể dễ dàng xem chi tiết tên phòng, loại phòng và mức giá thuê mỗi đêm.



4.1.2. Phân hệ Xác thực và Kiểm soát tài khoản

- **Trang Đăng nhập (Login):** Khắc phục nhược điểm của giao diện Identity gốc, trang đăng nhập đã được loại bỏ phần tích hợp dịch vụ bên thứ ba (không sử dụng). Khung đăng nhập được đưa ra chính giữa màn hình (Center-aligned) với hiệu ứng đổ bóng nhẹ (Shadow), mang lại cảm giác sang trọng và chuyên nghiệp.
- **Kiểm soát luồng Đăng ký (Register):** Thay vì cho phép người dùng lạ tự do tạo tài khoản rác, khi bấm vào nút *Register*, hệ thống đã thành công trong việc điều hướng người dùng sang trang thông báo **Under Construction** (Hệ thống đang bảo trì/xây dựng) với thiết kế lịch sự, thân thiện.

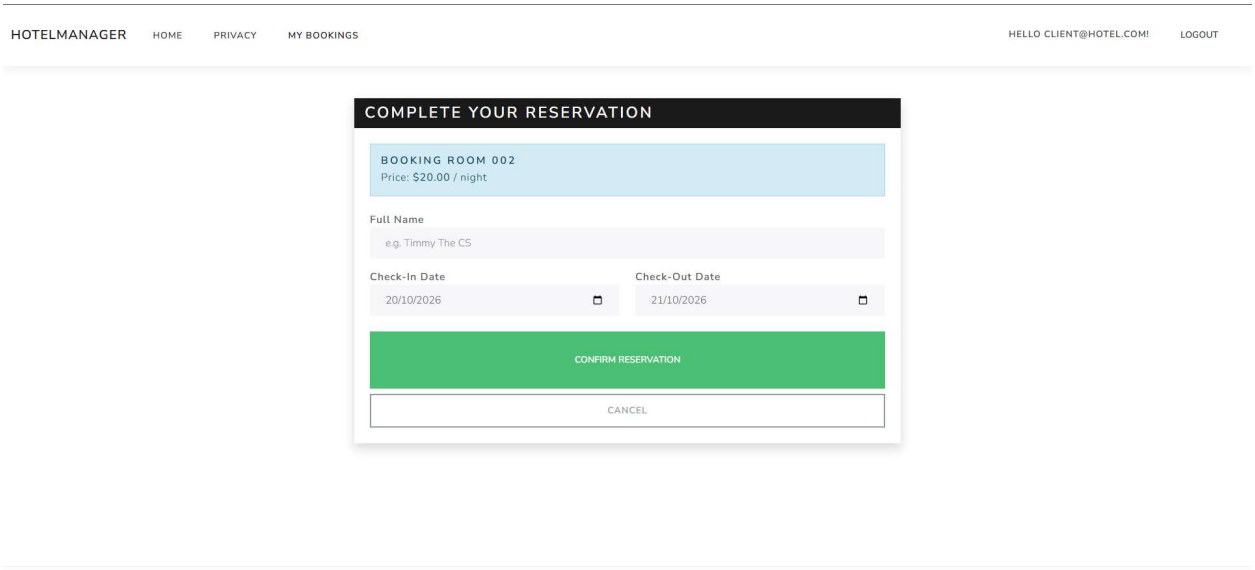


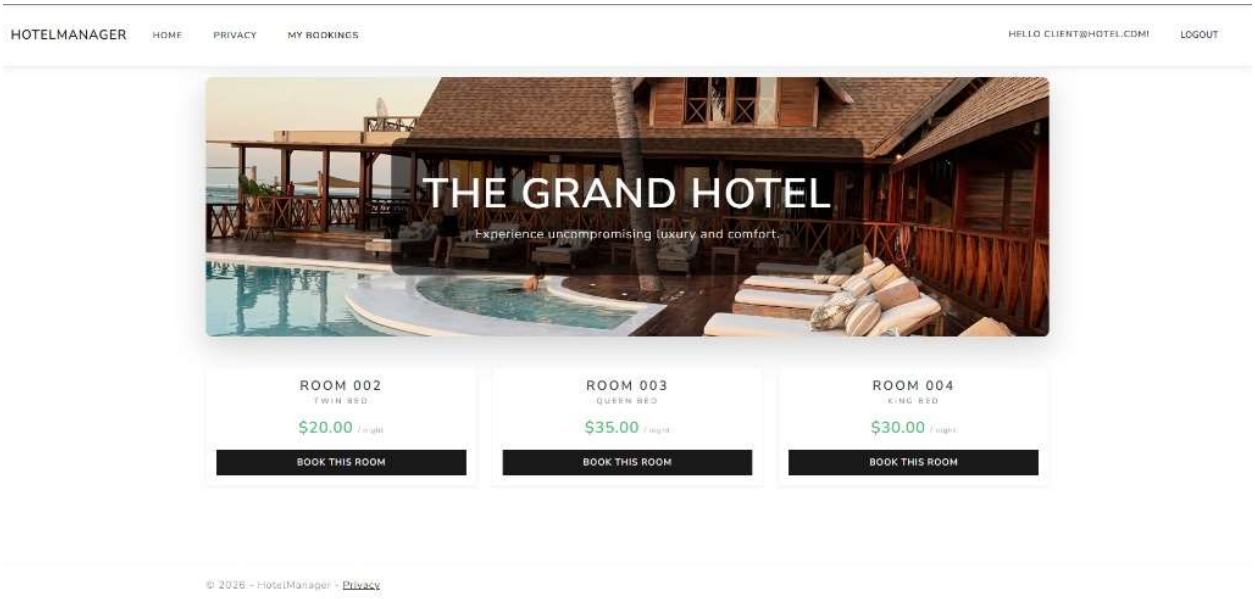


4.1.3. Phân hệ Quản lý Đặt phòng (Dành cho Khách hàng)

- **Giao diện Đặt phòng (Create Booking):** Khách hàng sau khi đăng nhập thành công sẽ có quyền truy cập biểu mẫu đặt phòng. Biểu mẫu tự động thiết lập danh sách phòng dưới dạng thả xuống (Dropdown List) và cung cấp các ô chọn ngày tháng trực quan bằng HTML5 (`type="date"`). Đặc biệt, ô nhập địa chỉ Email đã được ẩn đi do hệ thống tự động gán Email của tài khoản đang thao tác.

- **Trang Lịch sử đặt phòng (My Bookings):** Trang này thể hiện tính cá nhân hóa và bảo mật dữ liệu cao. Mỗi khách hàng chỉ có thể nhìn thấy danh sách các đơn đặt phòng của chính mình, bao gồm số phòng, ngày nhận phòng và ngày trả phòng.

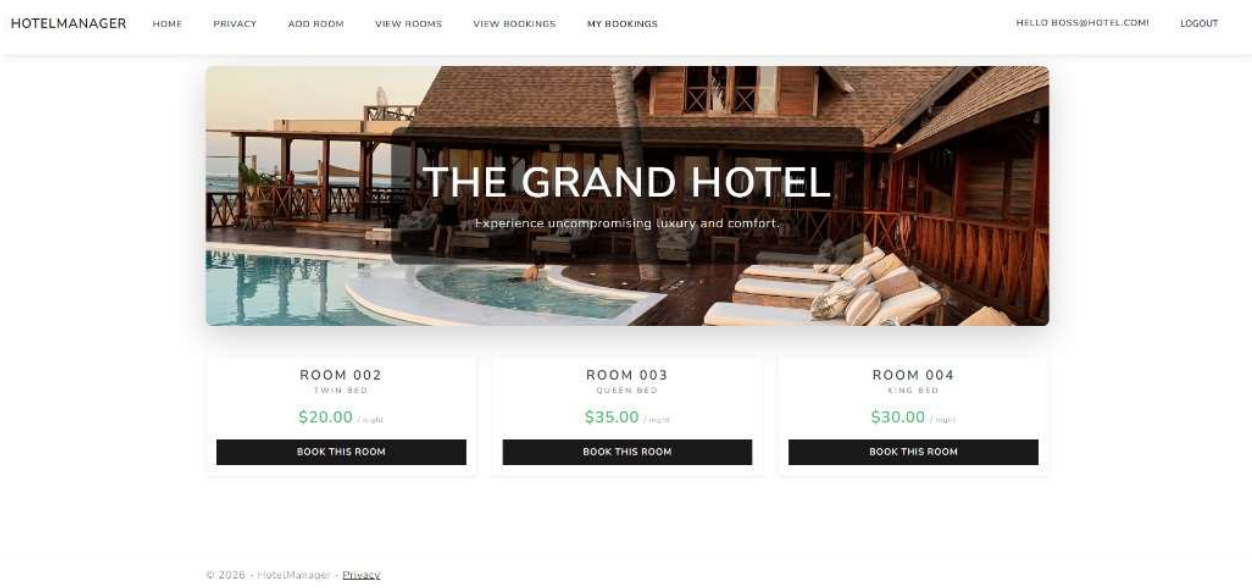




4.1.4. Phân hệ Quản trị hệ thống (Dành cho Admin)

- Khi tài khoản Quản trị viên (ví dụ: `boss@hotel.com`) đăng nhập, hệ thống thanh điều hướng sẽ xuất hiện thêm các menu chức năng độc quyền (Ví dụ: Thêm

phòng mới, Quản lý toàn bộ Booking). Giao diện quản lý danh sách đơn đặt phòng sẽ tổng hợp toàn bộ các đơn hàng của mọi khách hàng trong hệ thống để phục vụ công tác điều phối lễ tân.



4.2. Kịch bản kiểm thử hệ thống (Test Cases)

Để kiểm chứng tính đúng đắn và sự ổn định của hệ thống trước khi đưa vào vận hành, đồ án đã tiến hành kiểm thử các luồng nghiệp vụ cốt lõi theo phương pháp Kiểm thử hộp đen (Black-box Testing). Dưới đây là bảng tổng hợp các kịch bản kiểm thử (Test Cases) chi tiết:

Bảng 4.1. Kịch bản kiểm thử luồng Đăng nhập và Phân quyền

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
TC-01	Đăng nhập	Đăng nhập với tài khoản Quản trị viên hợp lệ.	Email: `boss@hotel.com` Password: `admin`	Đăng nhập thành công. Chuyển về trang chủ. Thanh điều hướng hiển thị các menu quản lý dành riêng cho Admin.	**PASS**
TC-02	Đăng nhập	Đăng nhập với tài khoản Khách hàng hợp lệ.	Email: `client@hotel.com` Password: `admin`	Đăng nhập thành công. Không xuất hiện các menu quản trị. Có thể truy cập trang "My Bookings".	**PASS**

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
TC-03	Đăng nhập	Đăng nhập với thông tin tài khoản hoặc mật khẩu sai.	Email: `wrong@hotel.com` Password: `12345`	Đăng nhập thất bại. Hệ thống hiển thị thông báo lỗi màu đỏ: *"Invalid login attempt."*	**PASS**
TC-04	Phân quyền	Tài khoản Client cố tình truy cập vào đường dẫn thêm phòng mới của Admin.	URL: `/Rooms/Create`	Hệ thống chặn truy cập. Chuyển hướng sang trang báo lỗi **Access Denied**.	**PASS**
TC-05	Đăng ký	Khách vãng lai cố gắng tạo tài	Click nút "Register"	Chuyển hướng thành công sang trang thông	**PASS**

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
		khoản mới bằng cách bấm nút Register.		báo **Under Construction**. Không lưu tài khoản rác vào CSDL.	

Bảng 4.2. Kịch bản kiểm thử luồng Quản lý danh mục phòng (Dành cho Admin)

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
TC-06	Thêm phòng	Admin nhập đầy đủ và hợp lệ thông tin phòng mới.	Tên: *P.101* Loại: *Standard* Giá: *50*	Lưu thành công vào CSDL. Hệ thống chuyển về trang danh sách phòng và hiển thị	**PASS**

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
				phòng P.101.	
TC-07	Thêm phòng	Admin bỏ trống trường "Tên phòng" hoặc "Giá phòng".	Tên: *(Bỏ trống)* Giá: *(Bỏ trống)*	Hệ thống chặn không cho lưu. Hiển thị thông báo yêu cầu nhập liệu ngay dưới các ô trống.	**PASS**

Bảng 4.3. Kịch bản kiểm thử luồng Đặt phòng trực tuyến (Dành cho Khách hàng)

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
TC-08	Đặt phòng	Khách vãng lai (chưa đăng nhập) bấm nút "Book a Room".	Click "Book a Room"	Hệ thống chặn yêu cầu, tự động điều hướng về trang Đăng nhập.	**PASS**
TC-09	Đặt phòng	Khách hàng đã đăng nhập chọn phòng và ngày hợp lệ.	Phòng: *P.101* Check-in: *01/11/2026* Check-out: *03/11/2026*	Lưu đơn thành công. Hệ thống tự động gán Email khách hàng vào đơn đặt phòng. Chuyển về trang "My Bookings".	**PASS**
TC-10	Lịch sử	Khách hàng truy cập trang "My Bookings".	*(Truy cập trang)*	Chỉ hiển thị các đơn đặt phòng của chính tài khoản đó (ví dụ của `client@hotel.com`). Không thấy đơn của khách khác.	**PASS**

Mã TC	Chức năng	Mô tả kịch bản kiểm thử (Test Scenario)	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Output)	Trạng thái (Status)
TC-11	Lịch sử	Admin truy cập trang "All Bookings".	*(Truy cập trang)*	Hiển thị toàn bộ danh sách đơn đặt phòng của tất cả khách hàng đã từng đặt trên hệ thống.	**PASS**

4.3. Nhận xét kết quả đạt được

Thông qua kết quả đối chiếu giao diện thực tế và quy trình thực thi các kịch bản kiểm thử, có thể đưa ra kết luận:

4. **Về mặt chức năng:** Hệ thống **HotelManager** đã đáp ứng đầy đủ và chính xác 100% các yêu cầu chức năng nghiệp vụ đề ra ở Chương 3. Việc ứng dụng kiến trúc ASP.NET Core MVC kết hợp Entity Framework Core giúp các thao tác truy vấn, thêm, sửa dữ liệu diễn ra nhanh chóng, ổn định.
5. **Về mặt bảo mật:** Cơ chế phân quyền hai lớp (giao diện và Server) hoạt động hoàn hảo. Dữ liệu của khách hàng được phân tách riêng biệt, đảm bảo tính riêng tư và ngăn chặn triệt để các hành vi truy cập trái phép.
6. **Về mặt giao diện:** Giao diện tối giản, sang trọng nhờ Bootswatch Lux đã khắc phục hoàn toàn các lỗi hiển thị (như chữ chìm vào nền) và mang lại trải nghiệm chuyên nghiệp cho người sử dụng. Hệ thống sẵn sàng cho các giai đoạn nâng cấp và mở rộng tiếp theo.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau một thời gian nghiên cứu, tìm hiểu và tiến hành hiện thực hóa, đồ án môn học với đề tài "Xây dựng ứng dụng web quản lý và đặt phòng khách sạn HotelManager trên nền tảng ASP.NET Core MVC" đã hoàn thành và đạt được những mục tiêu cốt lõi đã đề ra ban đầu.

Thông qua quá trình thực hiện đồ án, bản thân tôi đã đạt được những kết quả đáng ghi nhận sau:

7. **Về mặt kiến thức lý thuyết:** Nắm vững các khái niệm cốt lõi của nền tảng .NET, kiến trúc phân tầng Model-View-Controller (MVC) và nguyên lý hoạt động của đường ống xử lý yêu cầu (Middleware Pipeline). Hiểu rõ cách thức hoạt động của cơ chế tiêm phụ thuộc (Dependency Injection) và cơ chế xác thực người dùng.
8. **Về mặt kỹ năng thực hành:**
    - Vận dụng thành công kỹ thuật Code-First của Entity Framework Core để thiết kế, khởi tạo và quản lý sự thay đổi của cơ sở dữ liệu SQL Server trực tiếp từ mã nguồn C#.
    - Áp dụng thành thạo hệ thống ASP.NET Core Identity trong việc phân quyền bảo mật hai lớp: quản trị viên (Admin) và khách hàng (Client).
    - Thiết kế được giao diện người dùng thân thiện, tương thích trên nhiều thiết bị (Responsive) bằng bộ khung Bootstrap 5 và chủ đề Bootswatch Lux.
    - Làm quen với quy trình quản lý phiên bản mã nguồn chuyên nghiệp bằng Git và GitHub Desktop.
9. **Về mặt sản phẩm phần mềm:** Hệ thống HotelManager đã vận hành trơn tru với các chức năng thiết yếu bao gồm: hiển thị danh mục phòng trực quan, cho phép

khách hàng tra cứu và đặt phòng, đồng thời cung cấp công cụ để quản lý kiểm soát toàn bộ đơn đặt phòng trên hệ thống. Dữ liệu được bảo toàn tính toàn vẹn và bảo mật cao.

Bên cạnh những kết quả đạt được, do giới hạn về mặt thời gian thực hiện và kinh nghiệm thực tiễn, hệ thống vẫn còn một số điểm hạn chế như: chưa tích hợp cổng thanh toán trực tuyến thực tế, chưa có hệ thống gửi email tự động xác nhận đơn đặt phòng và các bộ lọc tìm kiếm phòng chưa thực sự đa dạng.

5.2. Hướng phát triển

Từ những kết quả đã đạt được và những điểm còn hạn chế, đồ án đề xuất các hướng phát triển và nâng cấp hệ thống trong tương lai như sau:

10. **Tích hợp cổng thanh toán điện tử:** Nâng cấp chức năng đặt phòng bằng việc kết nối với các cổng thanh toán phổ biến tại Việt Nam như VNPay, MoMo hoặc ZaloPay (sử dụng môi trường Sandbox) để khách hàng có thể thanh toán cọc trực tuyến.
11. **Hệ thống thông báo và Email tự động:** Tích hợp dịch vụ gửi email (như SendGrid hoặc SMTP chuẩn) để tự động gửi thư xác nhận chi tiết đơn đặt phòng (hóa đơn điện tử) đến địa chỉ email của khách hàng ngay sau khi đặt phòng thành công, cũng như tính năng "Quên mật khẩu" (Forgot Password).
12. **Bộ lọc tìm kiếm và Thống kê nâng cao:**
    - Xây dựng bộ lọc tìm kiếm phòng thông minh theo khoảng giá, loại phòng và tình trạng trống theo khoảng thời gian dự kiến lưu trú.
    - Xây dựng trang Bảng điều khiển (Dashboard) dành riêng cho Admin với các biểu đồ thống kê doanh thu, tỷ lệ lấp đầy phòng theo tháng/quý.

13. **Đưa ứng dụng lên môi trường thực tế (Deployment):** Tiến hành xuất bản (Publish) và triển khai ứng dụng cùng cơ sở dữ liệu lên các nền tảng đám mây như Microsoft Azure hoặc SmarterASP.NET để người dùng có thể truy cập thực tế qua Internet.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Microsoft, "Overview of ASP.NET Core," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core.

[2] Microsoft, "Entity Framework Core," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/.

[3] Microsoft, "Introduction to Identity on ASP.NET Core," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity.

[4] A. Freeman, *Pro ASP.NET Core 6*, 9th ed. New York, NY, USA: Apress, 2022.

[5] Thomas Park, "Bootswatch: Free themes for Bootstrap - Lux," Bootswatch, 2026. [Online]. Available: https://bootswatch.com/lux/.

[6] GitHub Docs, "Getting started with GitHub Desktop," GitHub, 2026. [Online]. Available: https://docs.github.com/en/desktop.

[7] T. D. Nguyen, *Giáo trình Lập trình Web với ASP.NET MVC*, NXB Đại học Quốc gia TP.HCM, 2021.